

Containerization

Docker and Kubernetes





Section 1

1. Introduction to Container
2. Basic of Docker
3. Docker components
4. Docker image
5. Docker container
6. Docker registry and image management
7. Dockerfile and docker compose

Section 2

1. Introduction to Kubernetes
2. K8s foundation (Pods, Deployment, Service)
3. K8s command with kubectl
4. Writing K8s manifest files
5. K8s components
5. Monitoring and logging

Container, Docker, Kubernetes

Build

Ship

Run

Monitoring and Observability

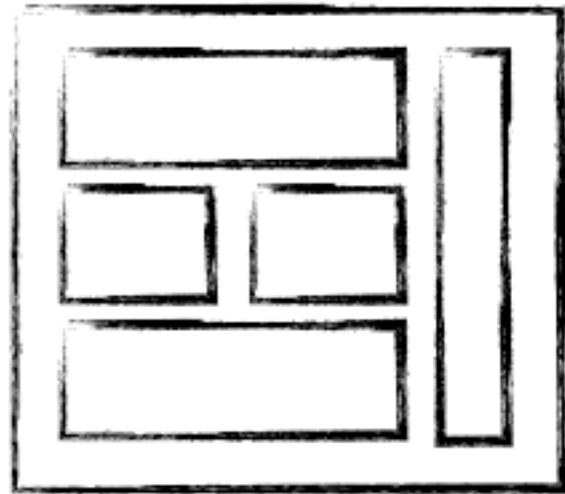
**[https://github.com/up1/
course-devops-workshop](https://github.com/up1/course-devops-workshop)**

Manage containers



Why docker ?

Software industry has changed !!

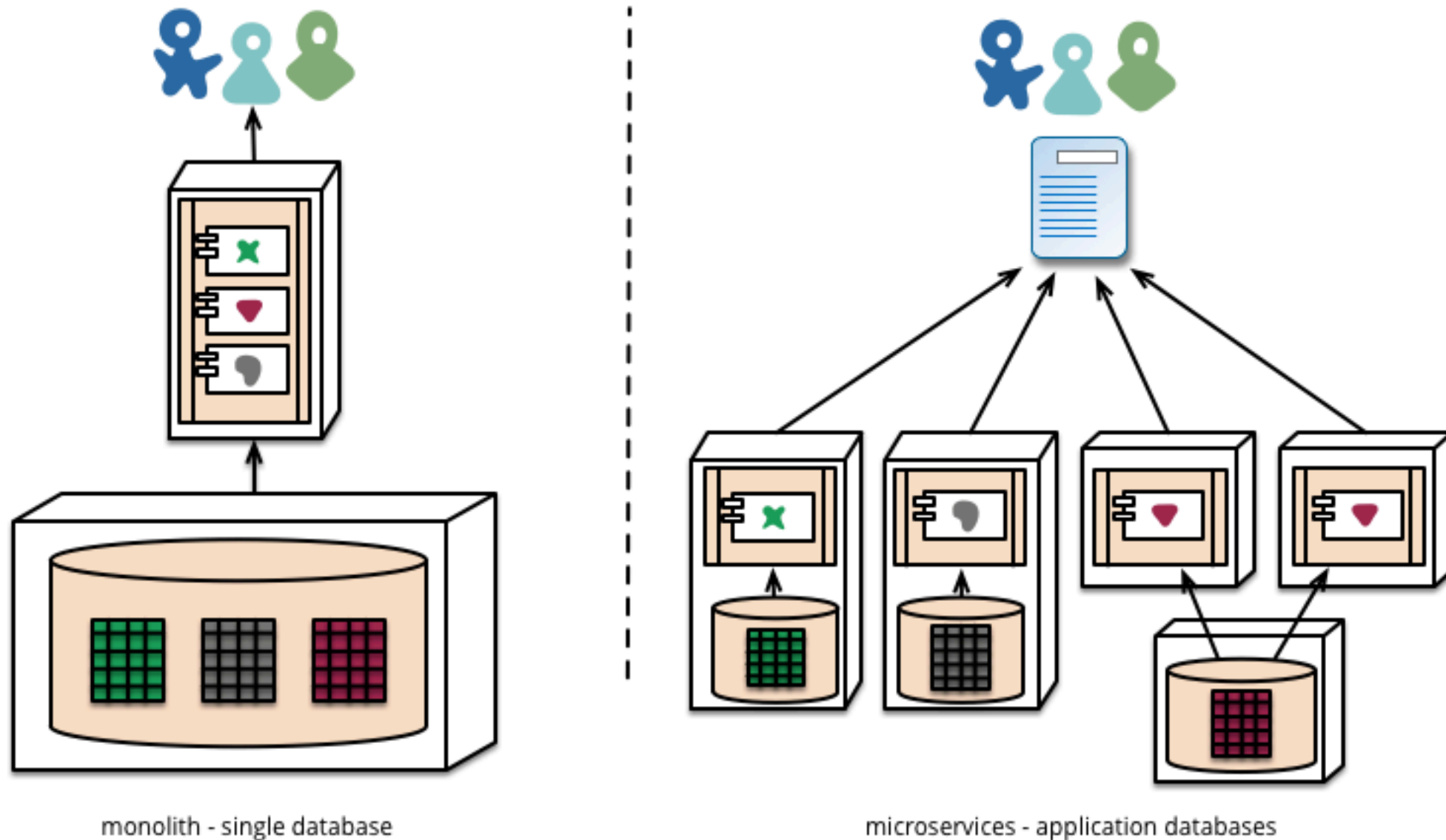


MONOLITHIC/LAYERED



MICRO SERVICES

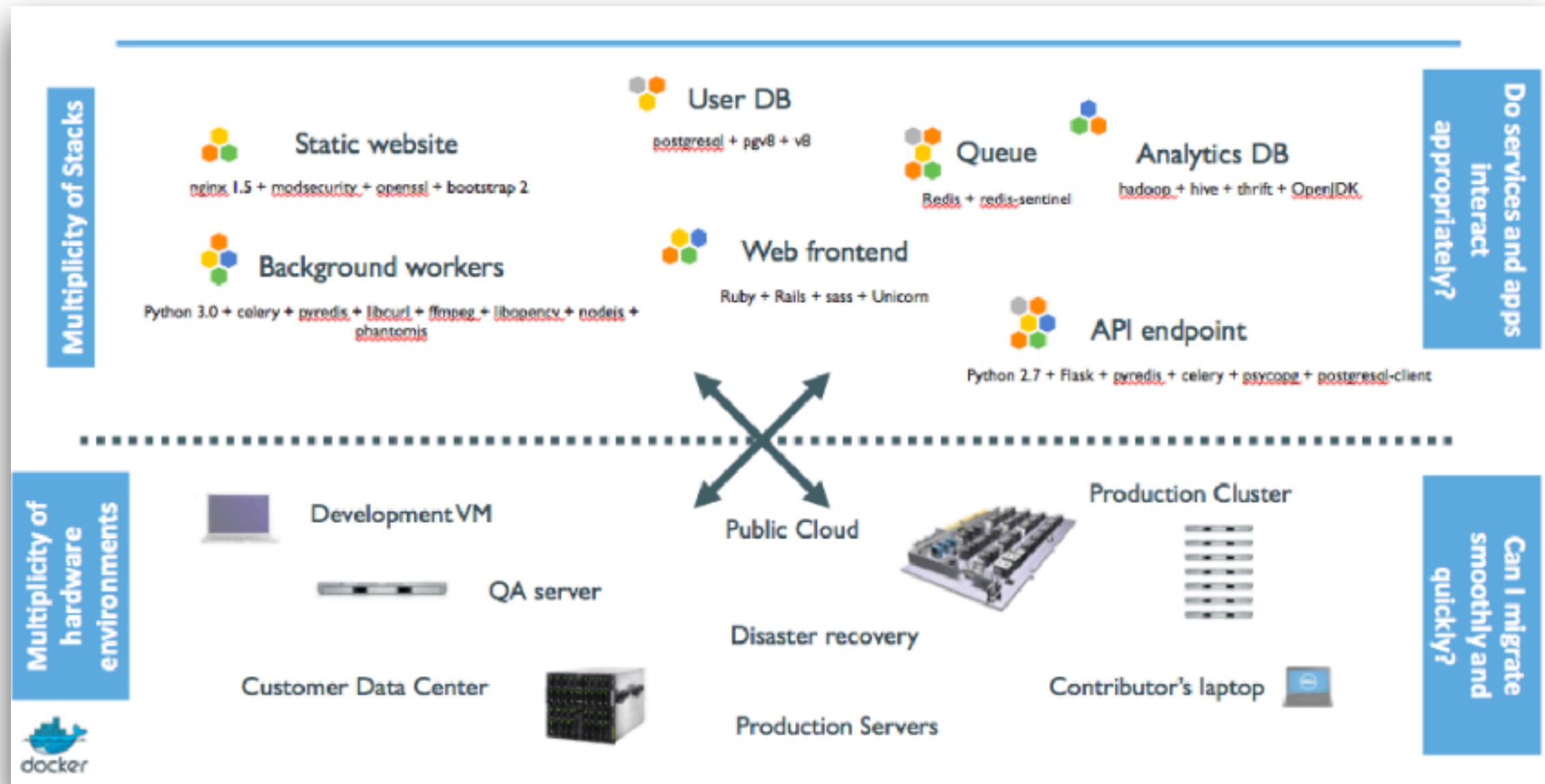
Microservice architecture



<https://martinfowler.com/articles/microservices.html>

Why docker ?

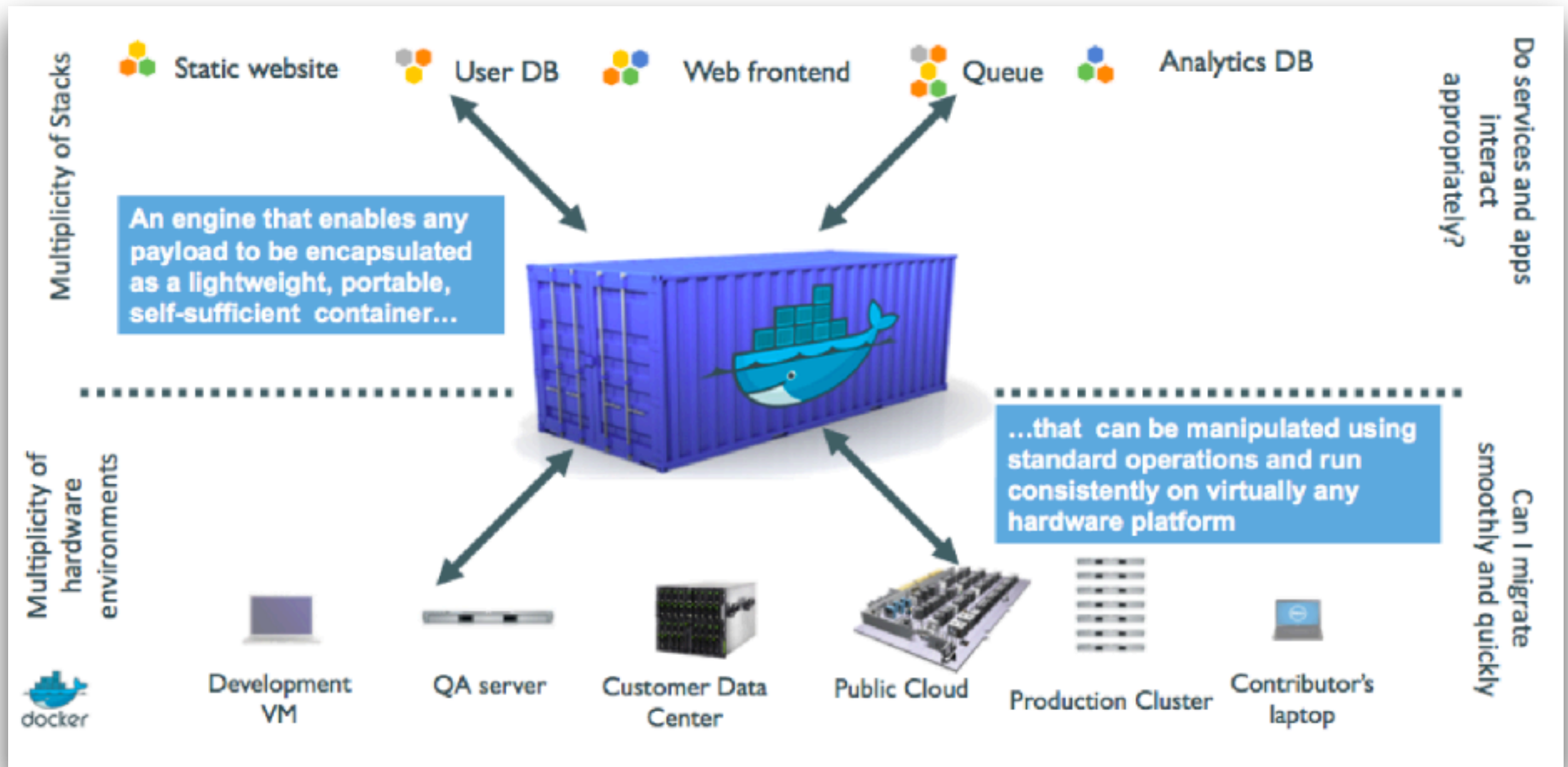
We have problem !!



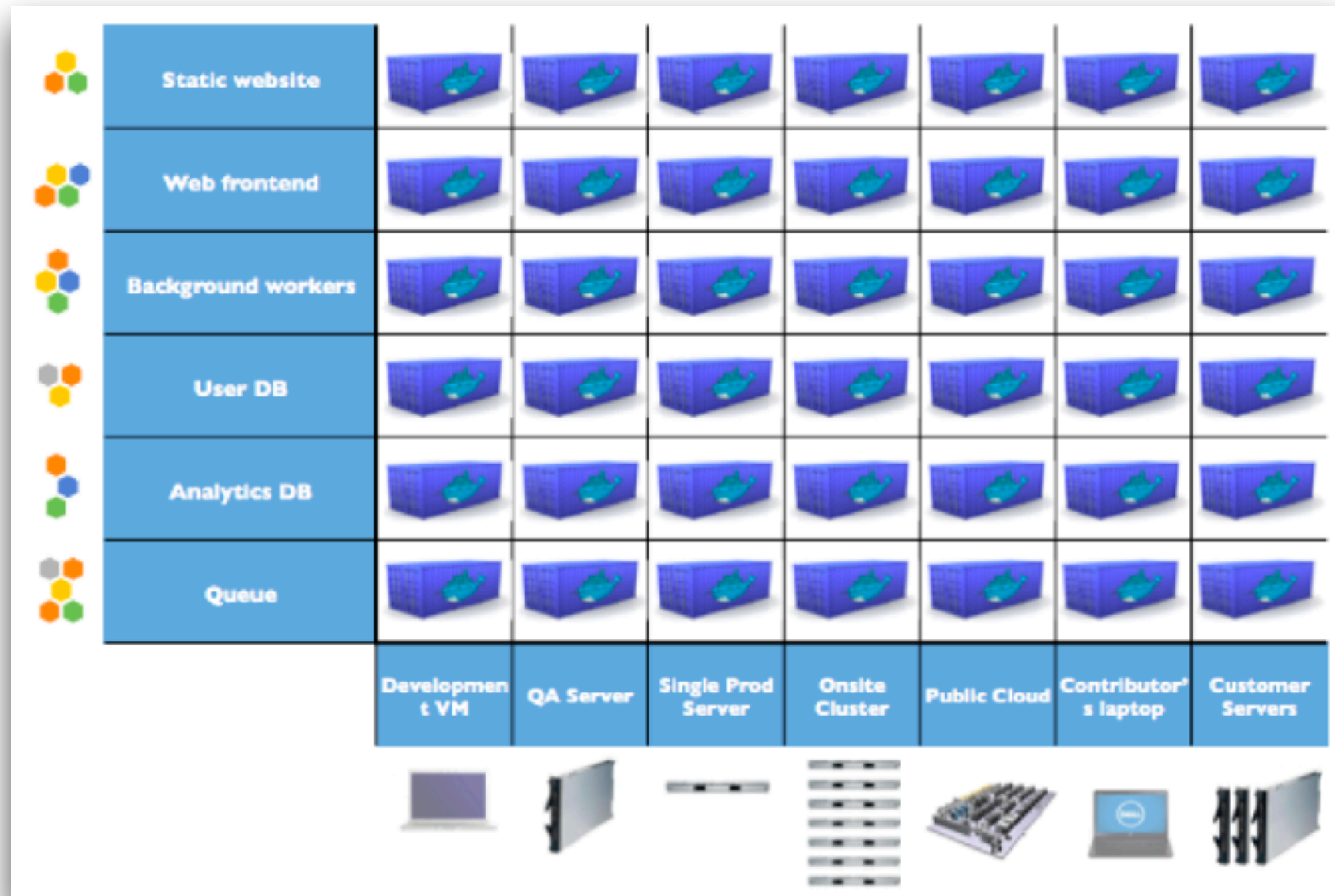
Problem ?

	Static website	?	?	?	?	?	?	?
	Web frontend	?	?	?	?	?	?	?
	Background workers	?	?	?	?	?	?	?
	User DB	?	?	?	?	?	?	?
	Analytics DB	?	?	?	?	?	?	?
	Queue	?	?	?	?	?	?	?
		Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
								

Solution ?

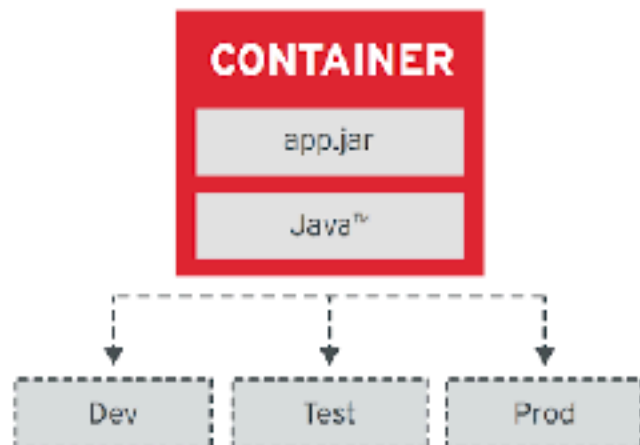


Solution ?



Container Design Principles

Image Immutability Principle



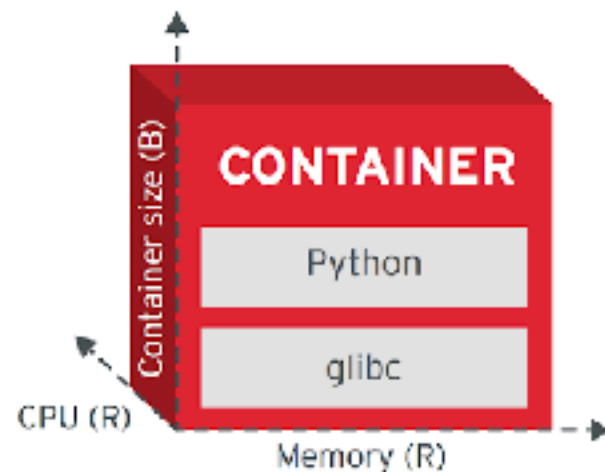
High Observability Principle



Process Disposability Principle



Runtime Confinement Principle



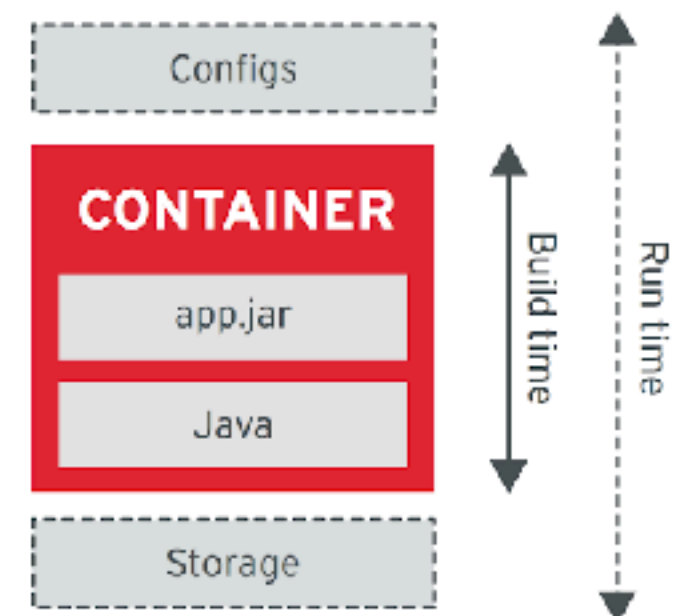
Lifecycle Conformance Principle



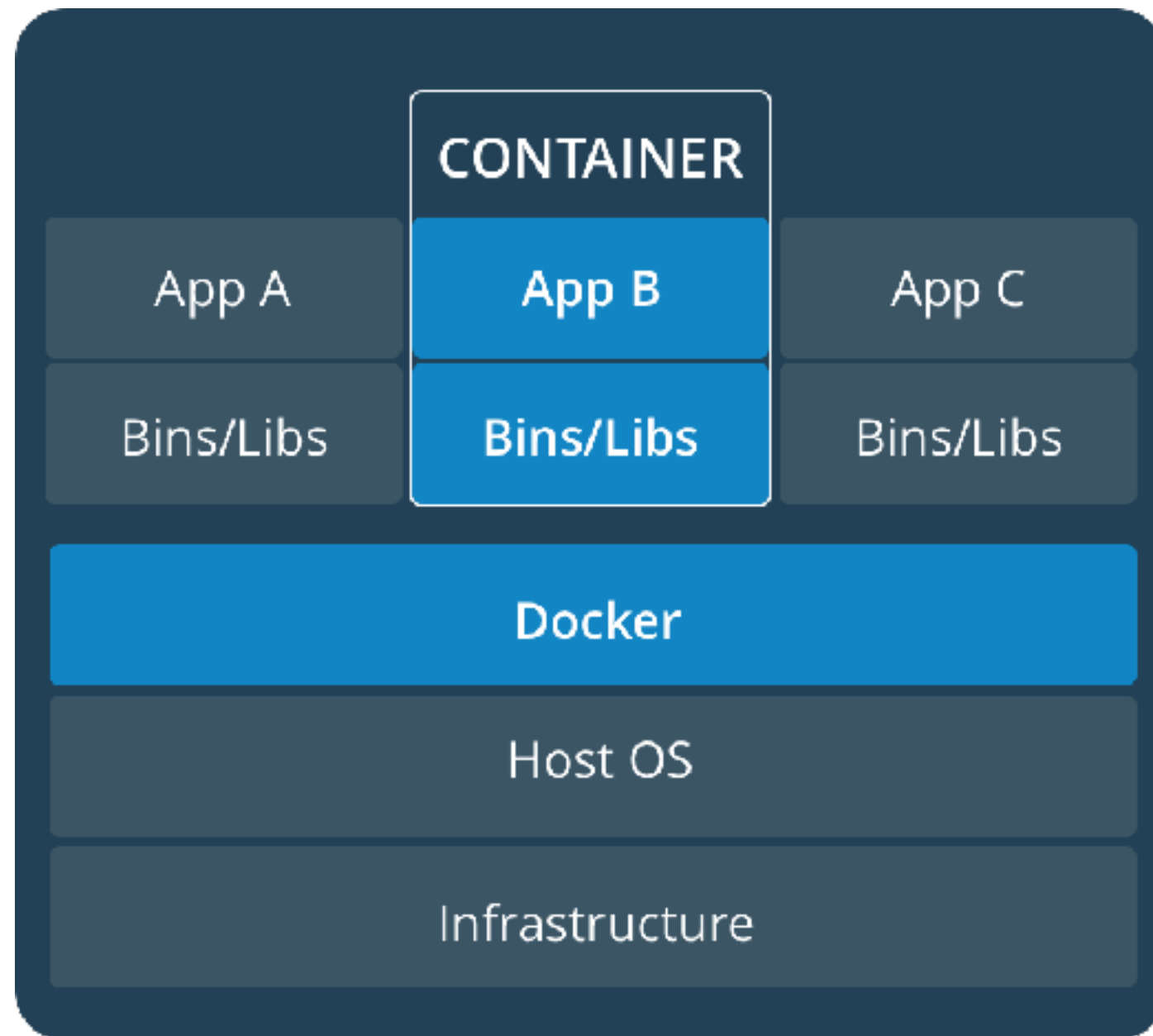
Single Concern Principle



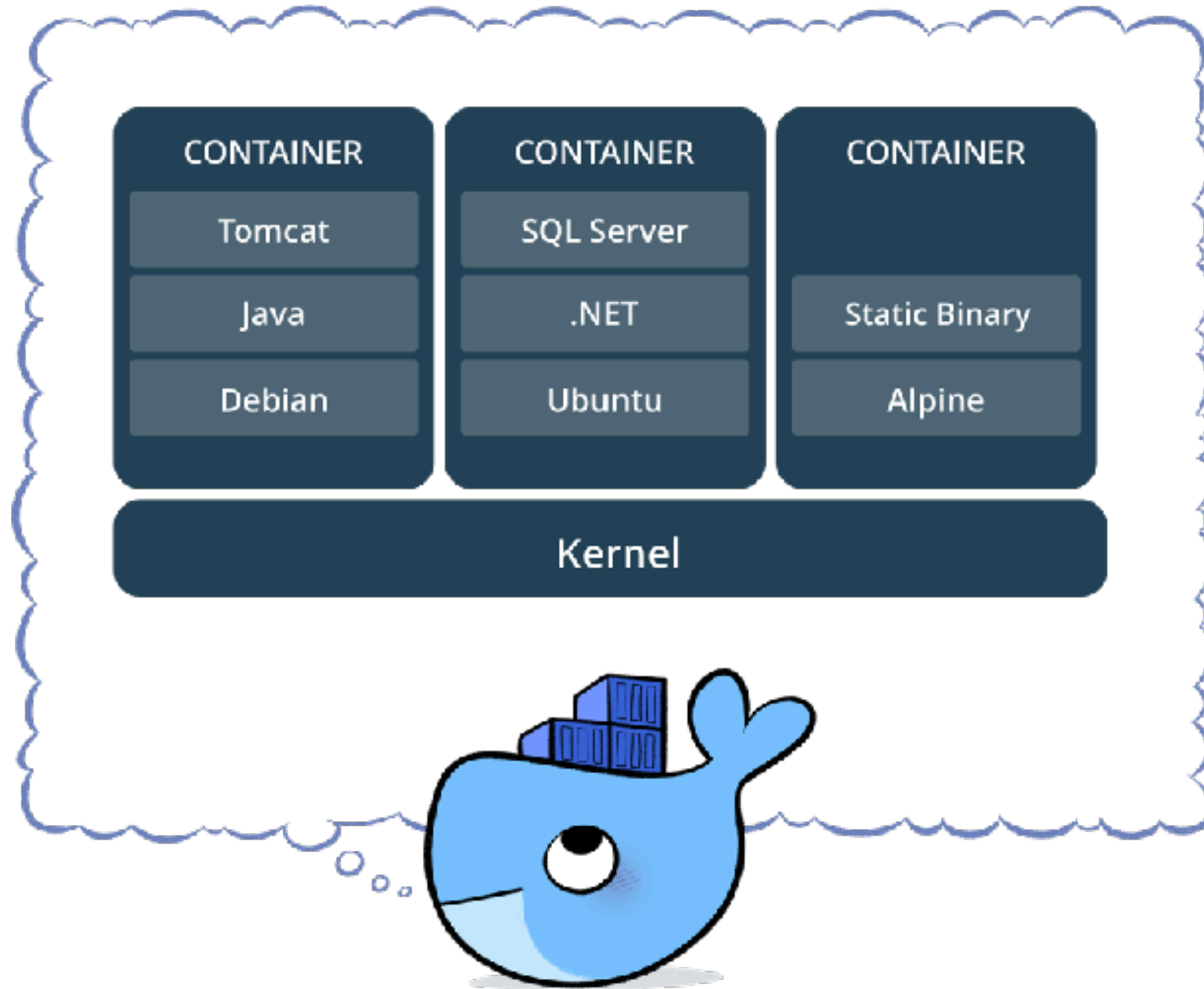
Self-Containment Principle



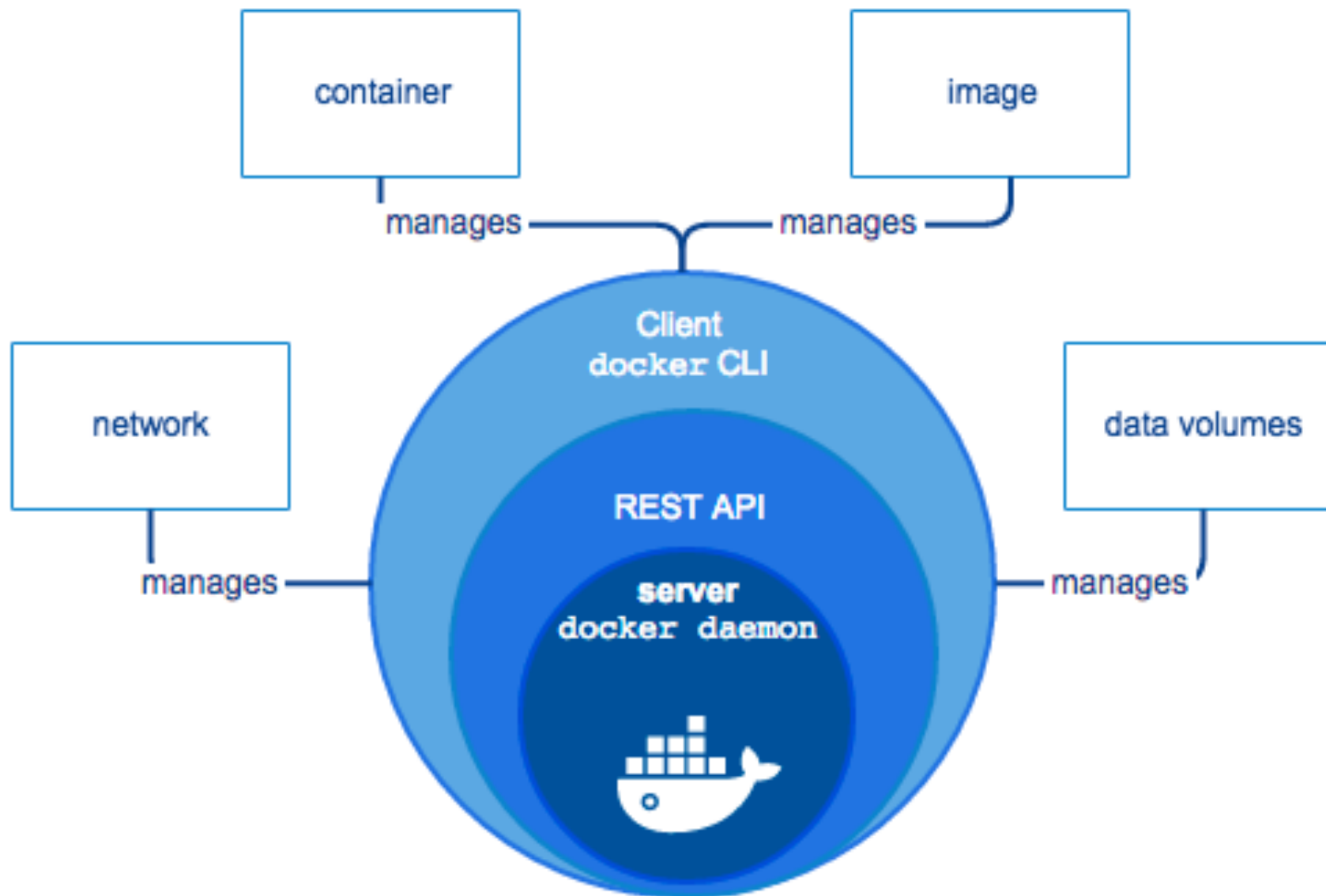
Container with Docker



Container with Docker



Basic of Docker



Basic of Docker

Image
Container
Dockerfile
Registry
Volume and network

Image vs Container

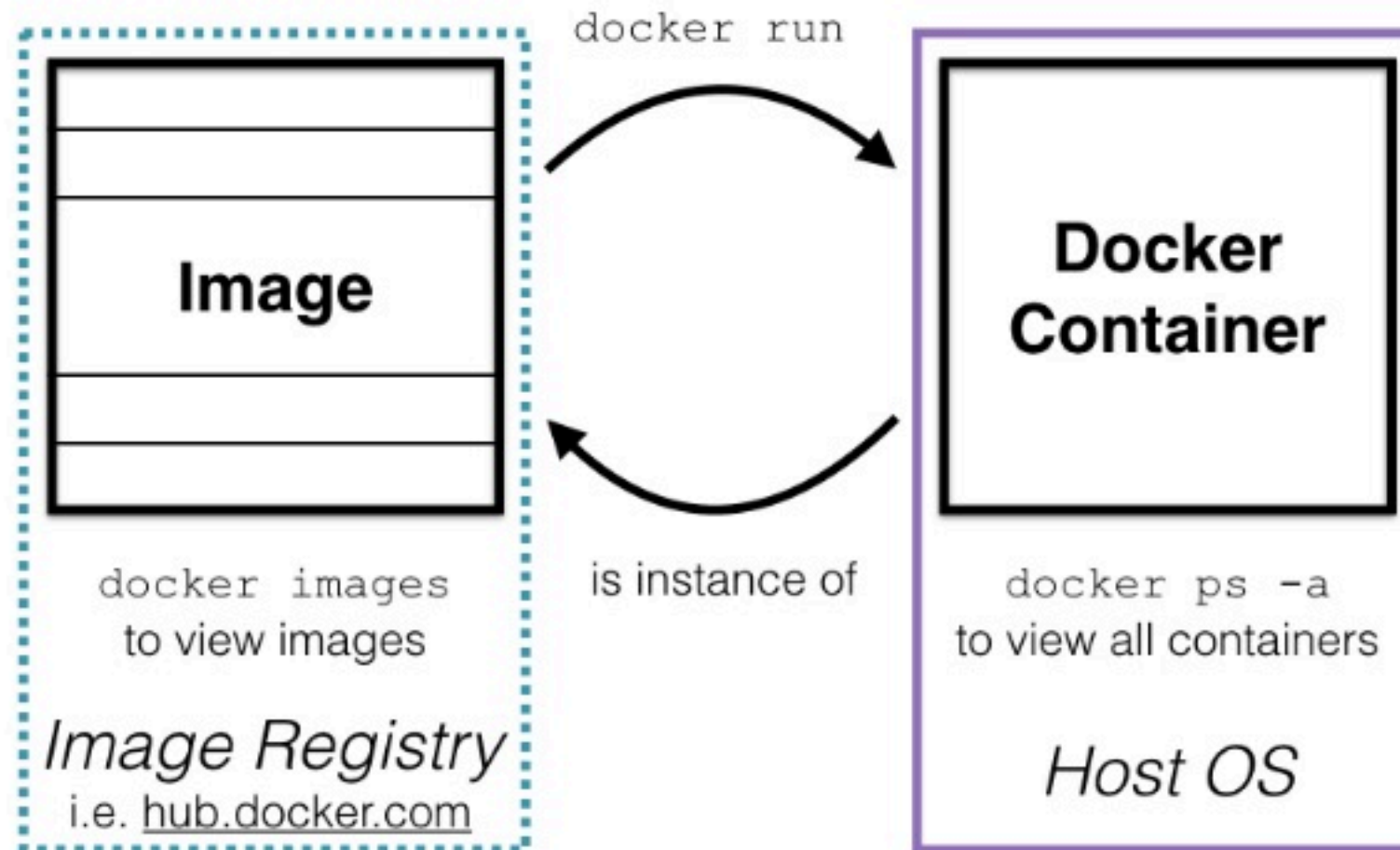


Image vs Container

Image like template/blueprint
Containers are created from image



Docker image

Collection of files and some meta data

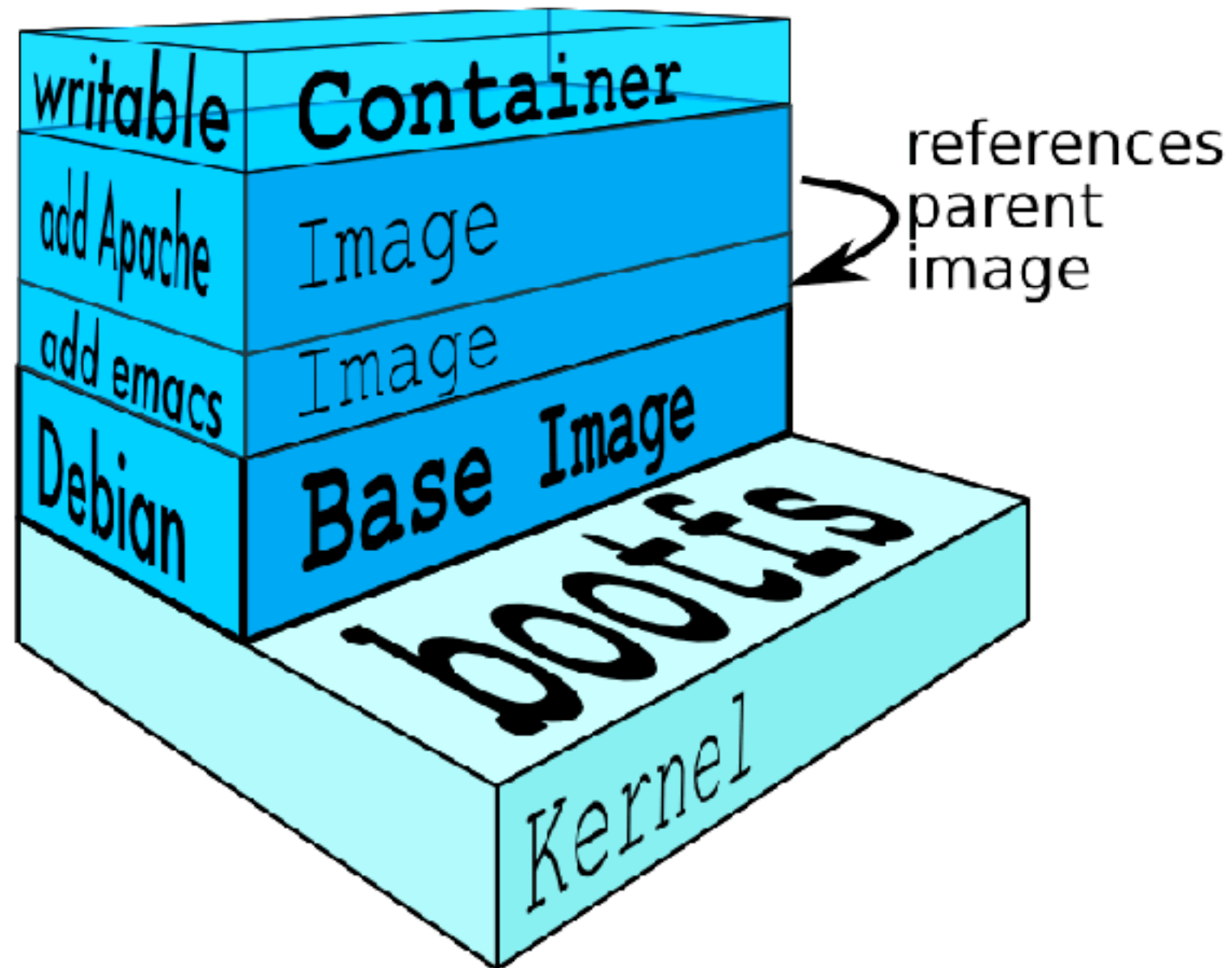
Made of **layers**

Each layer can add/change/remove files

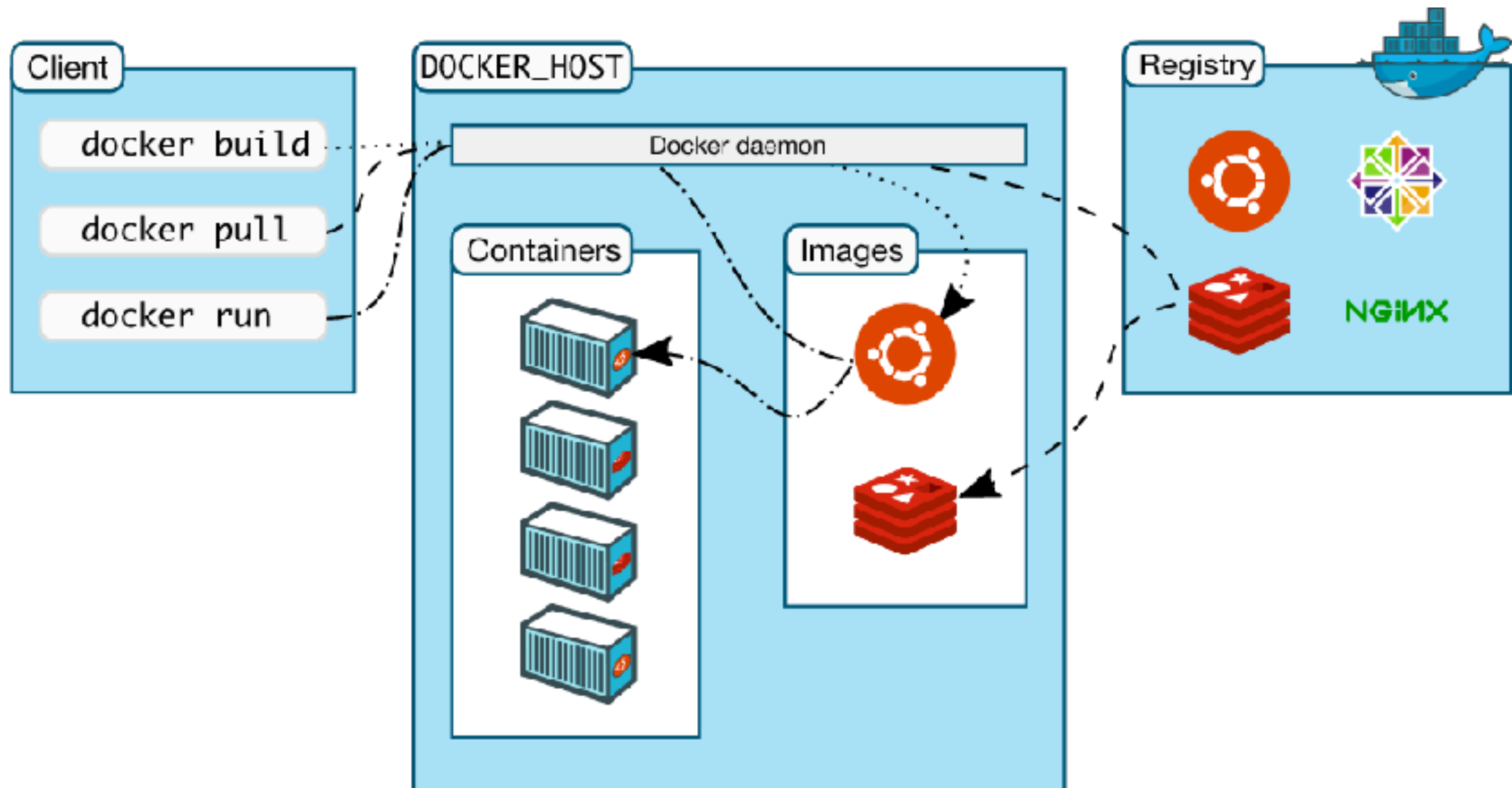
Image can share layers

Read-only file system

Docker image layer



How docker works ?



<https://docs.docker.com/get-started/overview/>

Docker commands

Manage image
Manage container
Manage system

Management commands

\$docker image <command>

\$docker container <command>

\$docker system <command>

Working with Nginx

OFFICIAL REPOSITORY

nginx 

Last pushed: 20 days ago

Repo Info [Tags](#)

Short Description

Official build of Nginx.

Docker Pull Command 

```
docker pull nginx
```

Full Description

Supported tags and respective **Dockerfile** links

- 1.11.10, mainline, 1, 1.11, latest ([mainline/jessie/Dockerfile](#))
- 1.11.10-alpine, mainline-alpine, 1-alpine, 1.11-alpine, alpine ([mainline/alpine/Dockerfile](#))
- 1.10.3, stable, 1.10 ([stable/jessie/Dockerfile](#))
- 1.10.3-alpine, stable-alpine, 1.10-alpine ([stable/alpine/Dockerfile](#))

https://hub.docker.com/_/nginx/

Pull image

\$docker image pull nginx:latest

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
693502eb7dfb: Pull complete
6decb850d2bc: Pull complete
c3e19f087ed6: Pull complete
Digest: sha256:52a189e49c0c797cfc5cbfe578c68c225d160fb13a42954144b29af3fe4fe335
Status: Downloaded newer image for nginx:latest
```

Pull image

```
$docker image pull nginx:latest
```



Tag

Image and Tag

Image can have **tags**
Tags define image variants
Default of tag is **:latest**

:latest tag can be updated frequently !!

Docker container

Foreground
Background
Interactive

Run with foreground

`$docker container run nginx`

Run with background

`$docker container run -d nginx`

-d = --detach

Run container in background and print container ID

Run with interactive

\$docker container run **-it** nginx bash

-i = --interactive

-t = --tty

Access to container

\$docker container **run -it**

\$docker container **exec -it**

Create Image from Dockerfile

Dockerfile

Build recipe for a Docker image

Contain series of instructions

Use **\$docker image build** command

<https://docs.docker.com/engine/reference/builder/>

Example

```
FROM ubuntu  
RUN apt-get update  
RUN apt-get install -y wget
```

Build image from Dockerfile

`$docker image build -t first_image .`

```
Sending build context to Docker daemon 2.048 kB
Step 1/3 : FROM ubuntu
----> 0ef2e08ed3fa
Step 2/3 : RUN apt-get update
----> Running in 6c598d2946b7
Get:1 http://archive.ubuntu.com/ubuntu xenial InF
Get:2 http://archive.ubuntu.com/ubuntu xenial-upd
Get:3 http://archive.ubuntu.com/ubuntu xenial-sec
Get:4 http://archive.ubuntu.com/ubuntu xenial/mai
Get:5 http://archive.ubuntu.com/ubuntu xenial/res
```

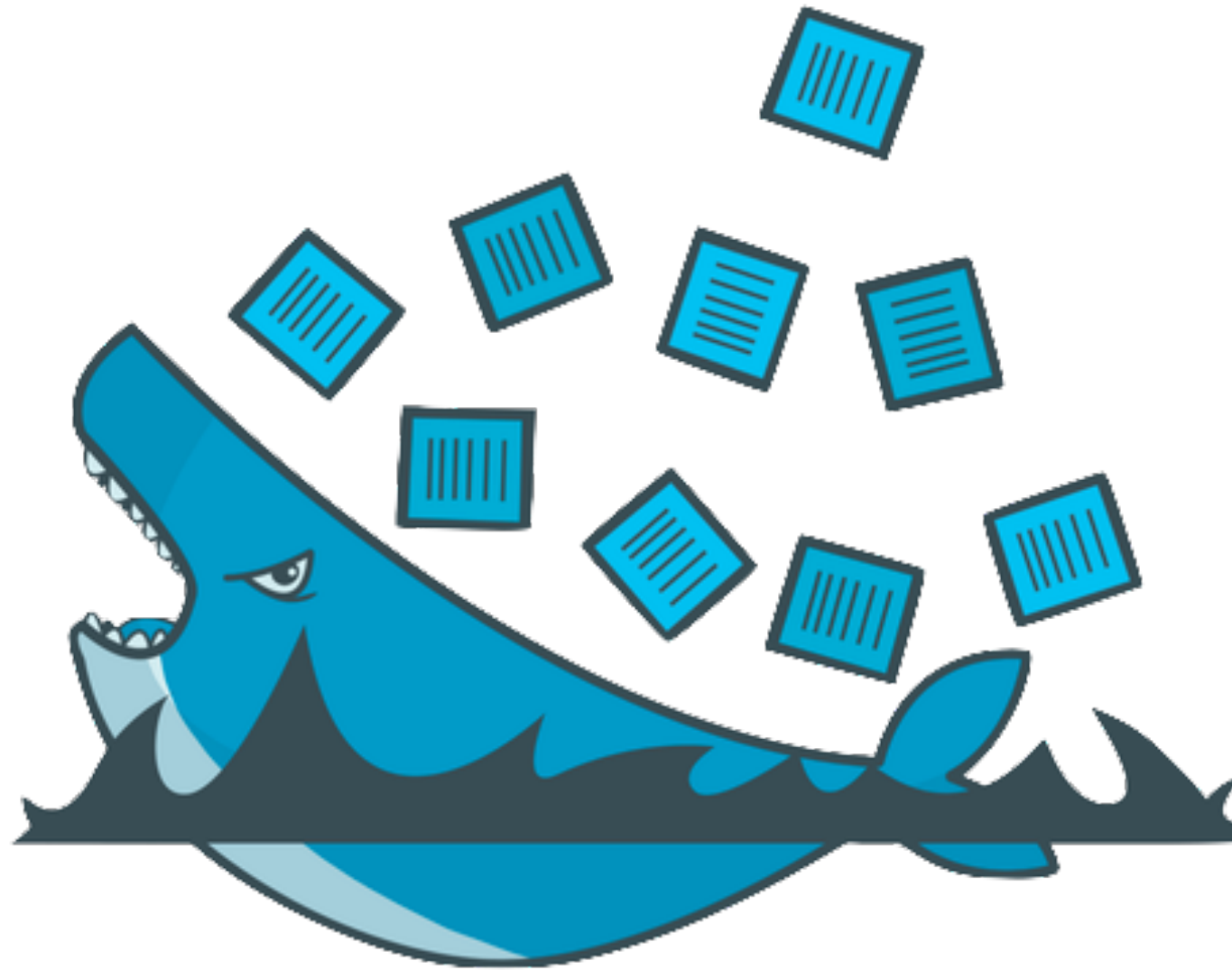

History of image

Show all layers of image

\$docker image **history** <image name>

IMAGE	CREATED	CREATED BY	SIZE
1813d5ecf658	4 minutes ago	/bin/sh -c apt-get install -y wget	7.35 MB
2930e9a322d6	5 minutes ago	/bin/sh -c apt-get update	40.1 MB
0ef2e08ed3fa	3 weeks ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0 B
<missing>	3 weeks ago	/bin/sh -c mkdir -p /run/systemd && echo '...	7 B
<missing>	3 weeks ago	/bin/sh -c sed -i 's/^#\s*\s*(deb.*universe\...	1.9 kB
<missing>	3 weeks ago	/bin/sh -c rm -rf /var/lib/apt/lists/*	0 B
<missing>	3 weeks ago	/bin/sh -c set -xe && echo '#!/bin/sh' >...	745 B
<missing>	3 weeks ago	/bin/sh -c #(nop) ADD file:efb254bc677d66d...	130 MB

Workshop



Working with Docker compose

<https://docs.docker.com/compose/>

Mutiple containers app!!

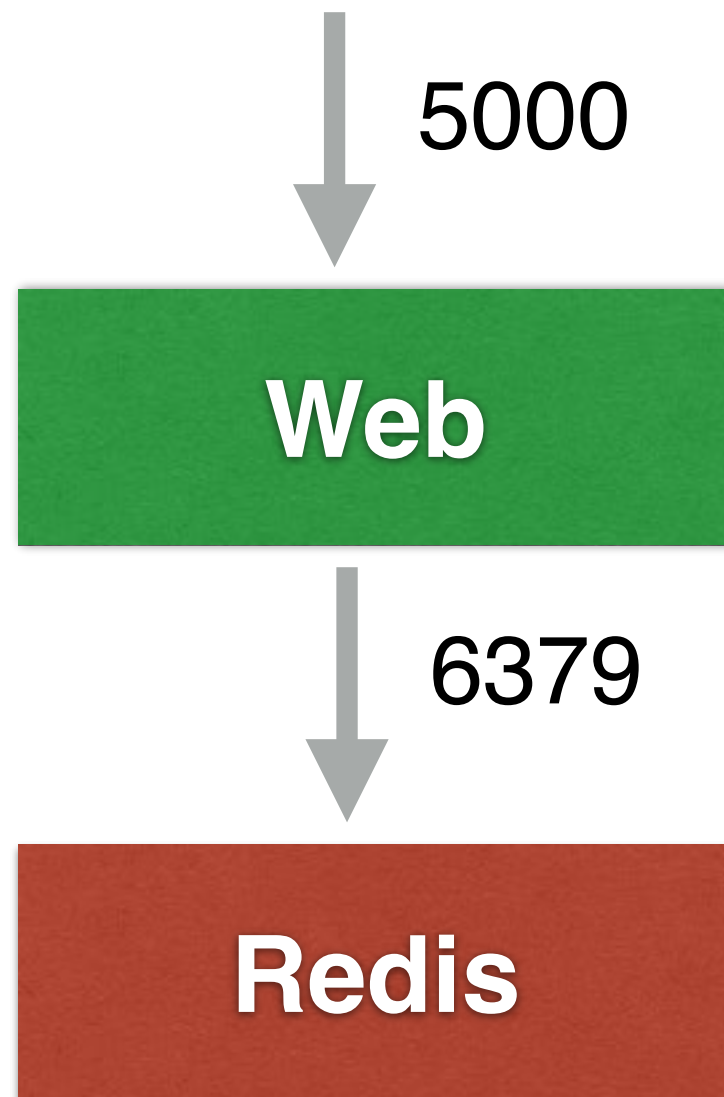
Difficult to create and manage !!



Mutiple containers app!!

- Build images from Dockerfile
- Pull images from Hub/private/cache
- Configuration and create containers
- Start and stop containers
- Stream their logs

Mutliple containers app!!



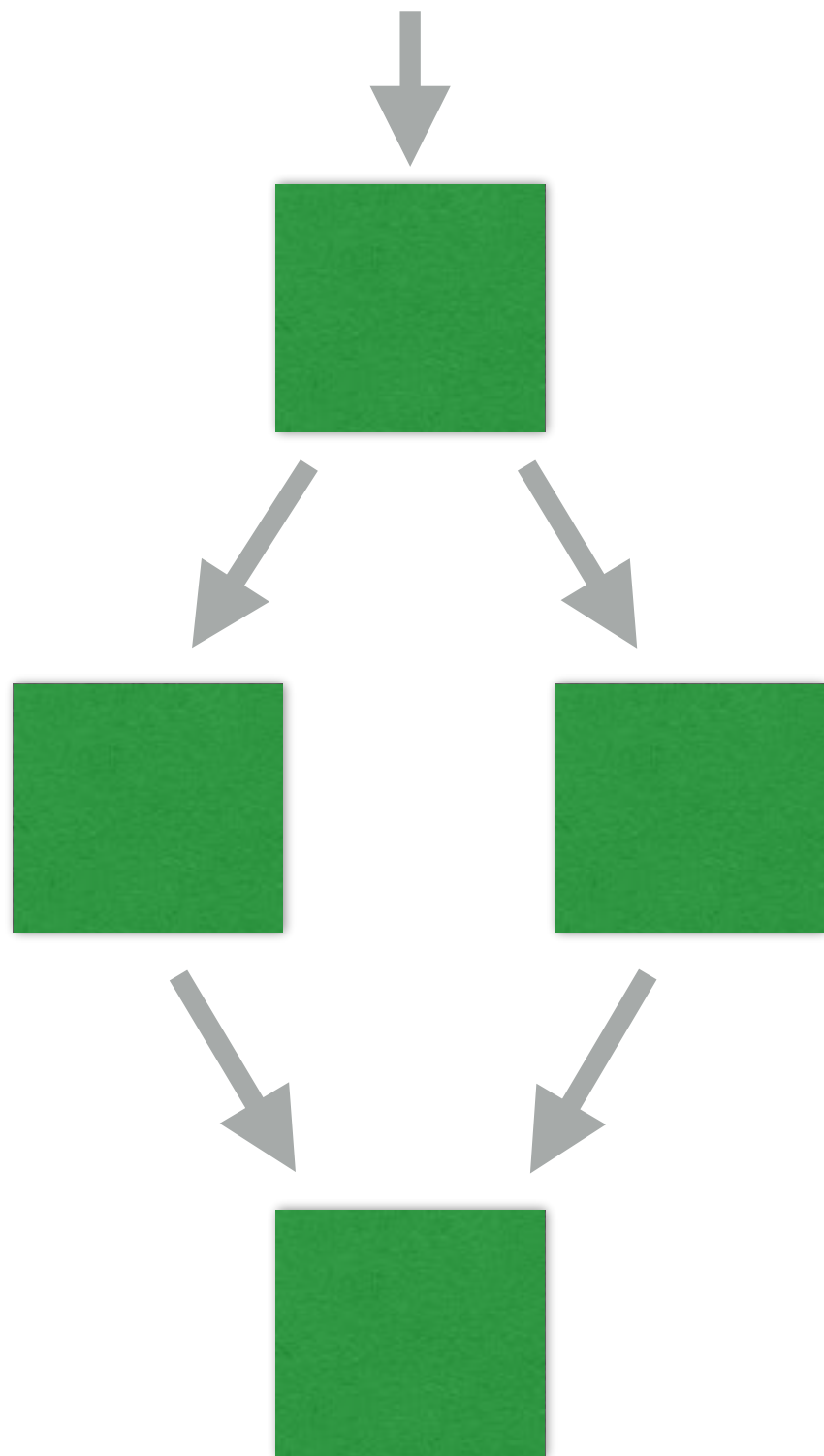
```
$docker pull redis:latest
```

```
$docker build -t web .
```

```
$docker run -d --name db redis  
redis-server --appendonly yes
```

```
$docker run -d --name web --link  
db:db -p 5000:5000 -e  
REDIS_HOST=db -v $(pwd):/code  
web
```

Mutliple containers app!!



\$docker pull

\$docker build

\$docker build

\$docker build

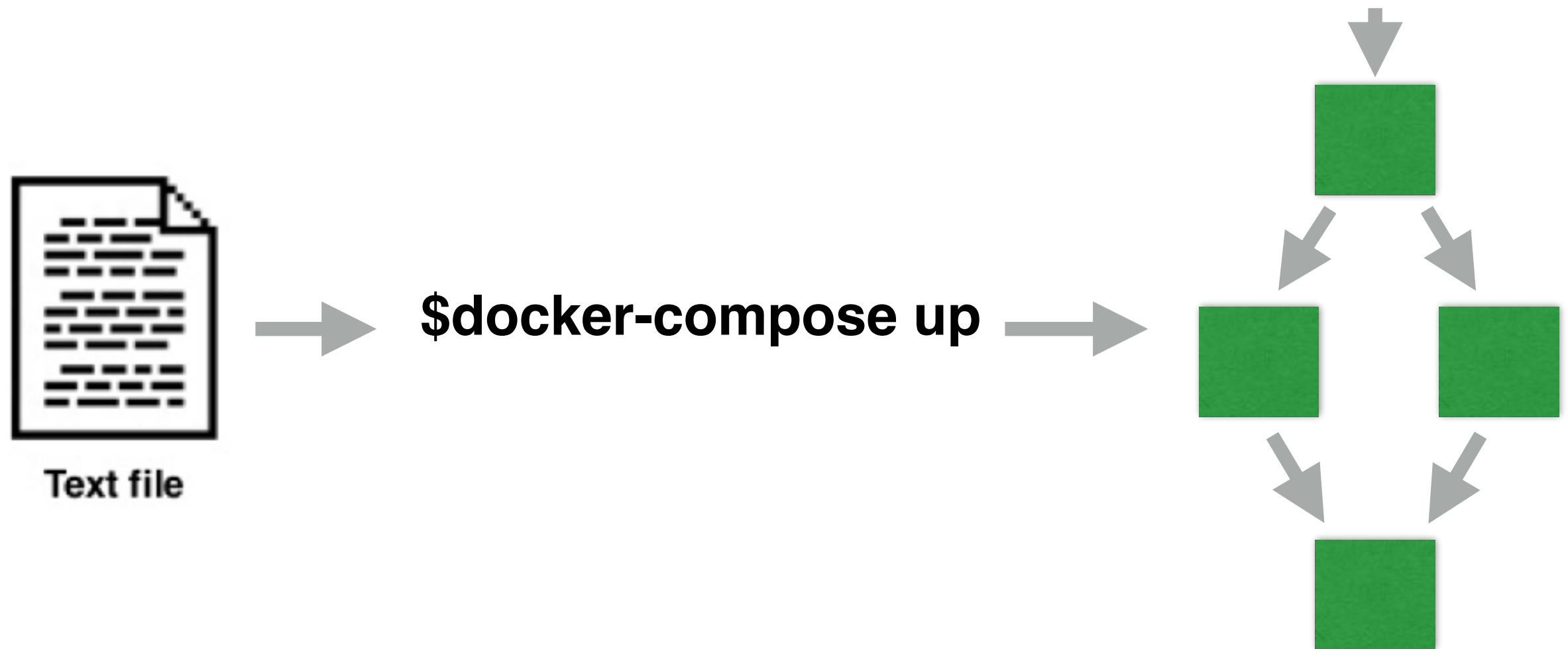
\$docker run

\$docker run

\$docker run

Docker compose

Running app in one command-line
Configuration in YAML file



docker-compose.yml

```
version: '3'
services:
  web:
    container_name: web
    build: .
    ports:
      - 80:5000
    environment:
      - REDIS_HOST=redis

  redis:
    container_name: redis
    image: redis
```


Build and run

`$docker-compose build`

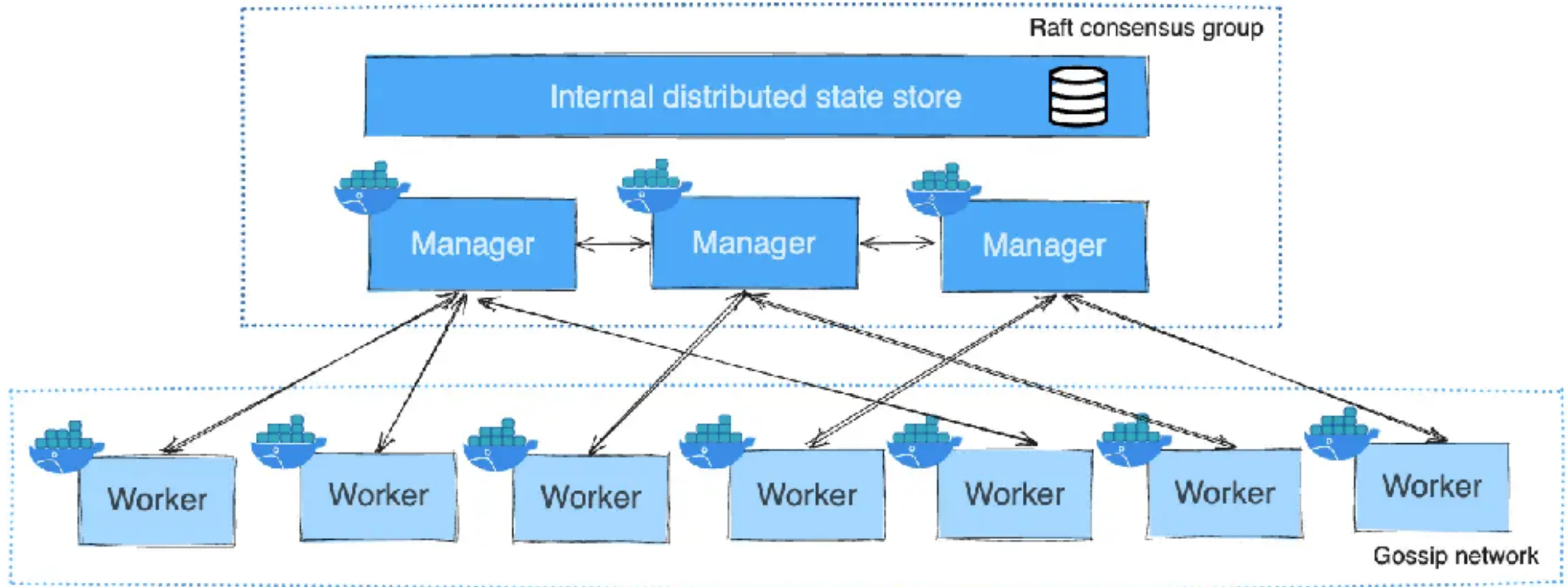
`$docker-compose up -d`

`$docker-compose logs --follow`

`$docker-compose ps`

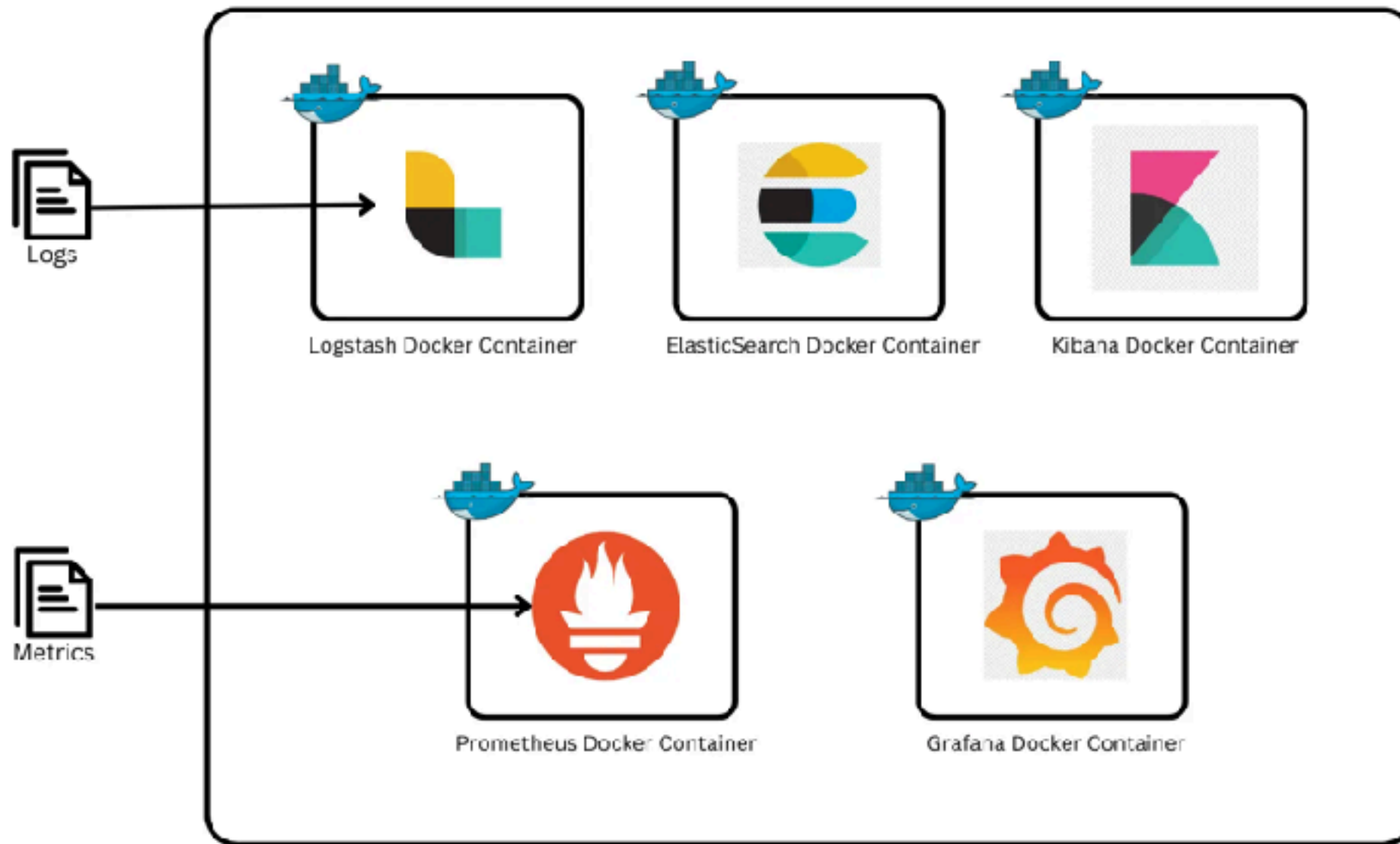
`$docker-compose down`

Docker Swarm



<https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/>

Monitoring with Docker



Workshop



Introduction



kubernetes

Why we need Kubernetes ?

Managing containers for production is challenging

Need something to manage beyond a container engine !!



Key capabilities was missing

Using multiple containers with shared resources

Monitoring running containers

Handling dead containers

Moving containers so utilization improves

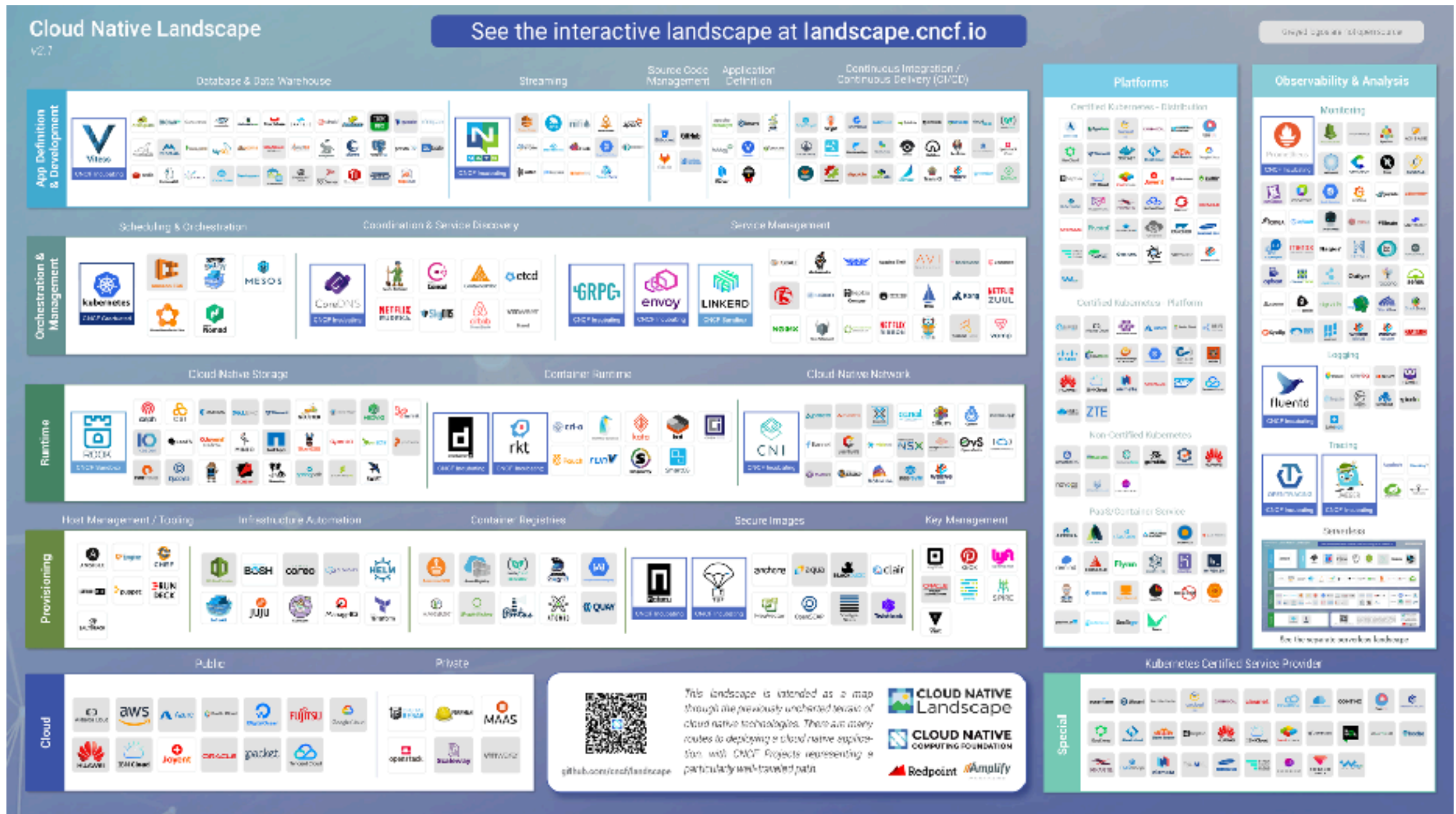
Key capabilities was missing

Autoscaling container instances to handle load

Making the container services easily accessible

Handling dead Connecting containers to a variety of external data sources

Cloud Native Landscape



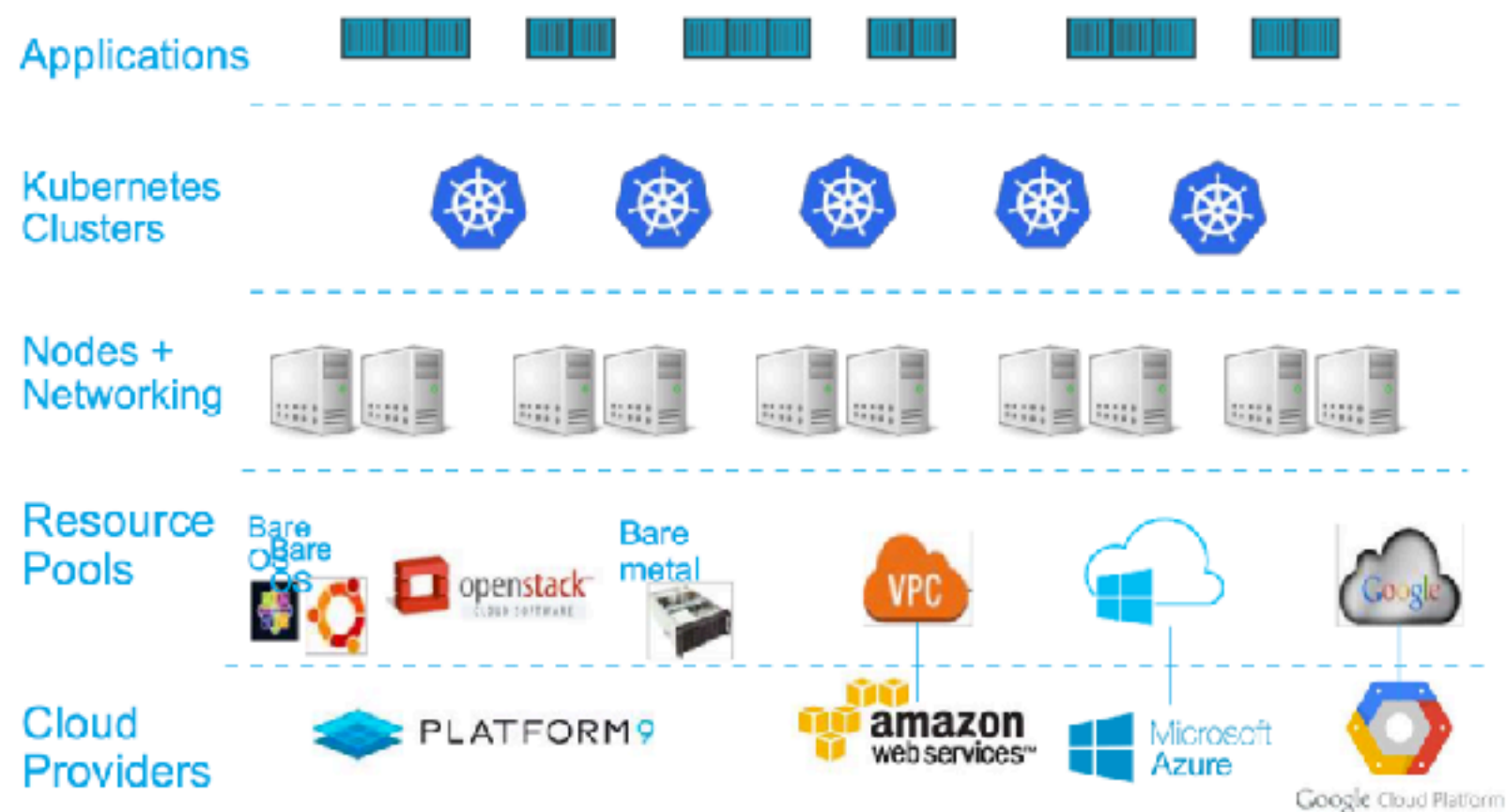
<https://github.com/cncf/landscape>

Write once, run anywhere

Eliminate infrastructure lock-in

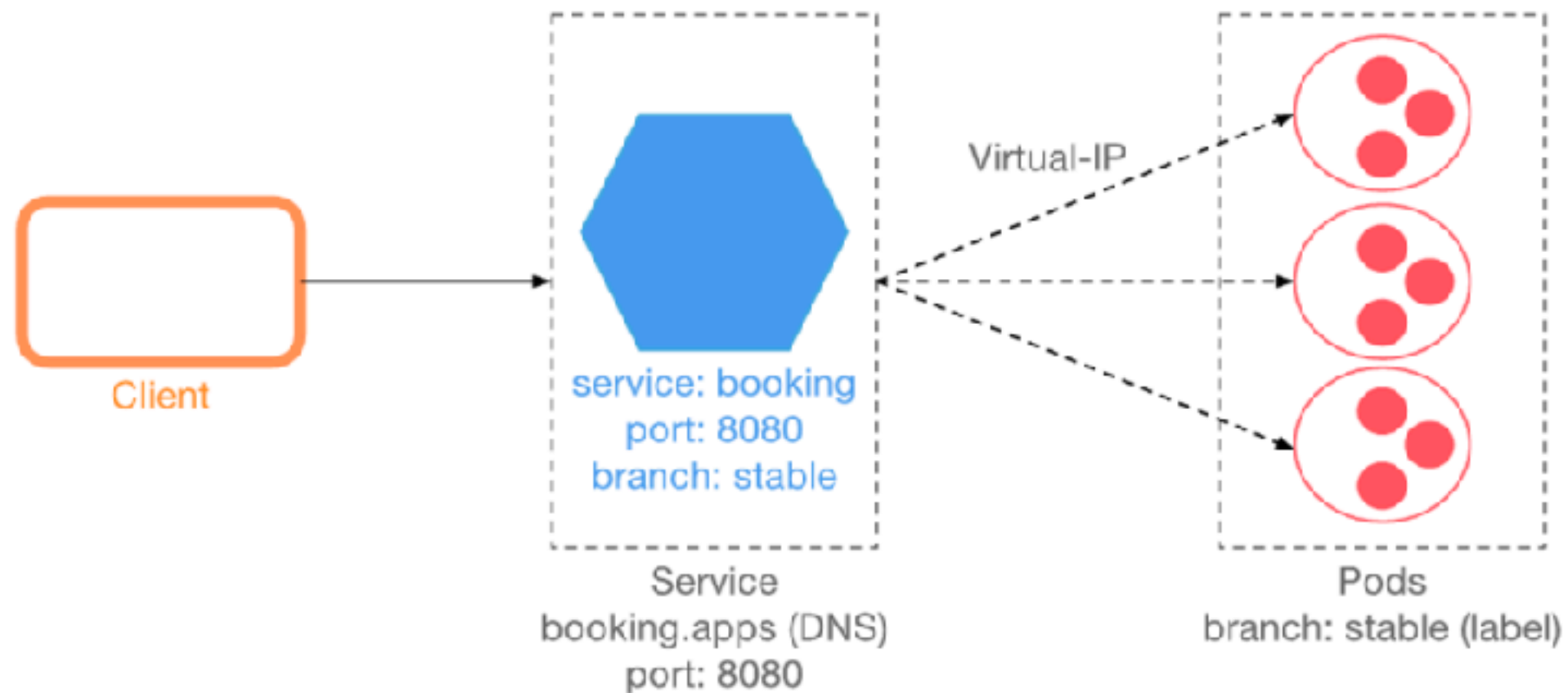
Use containers

Provides management for containers



Modular app design

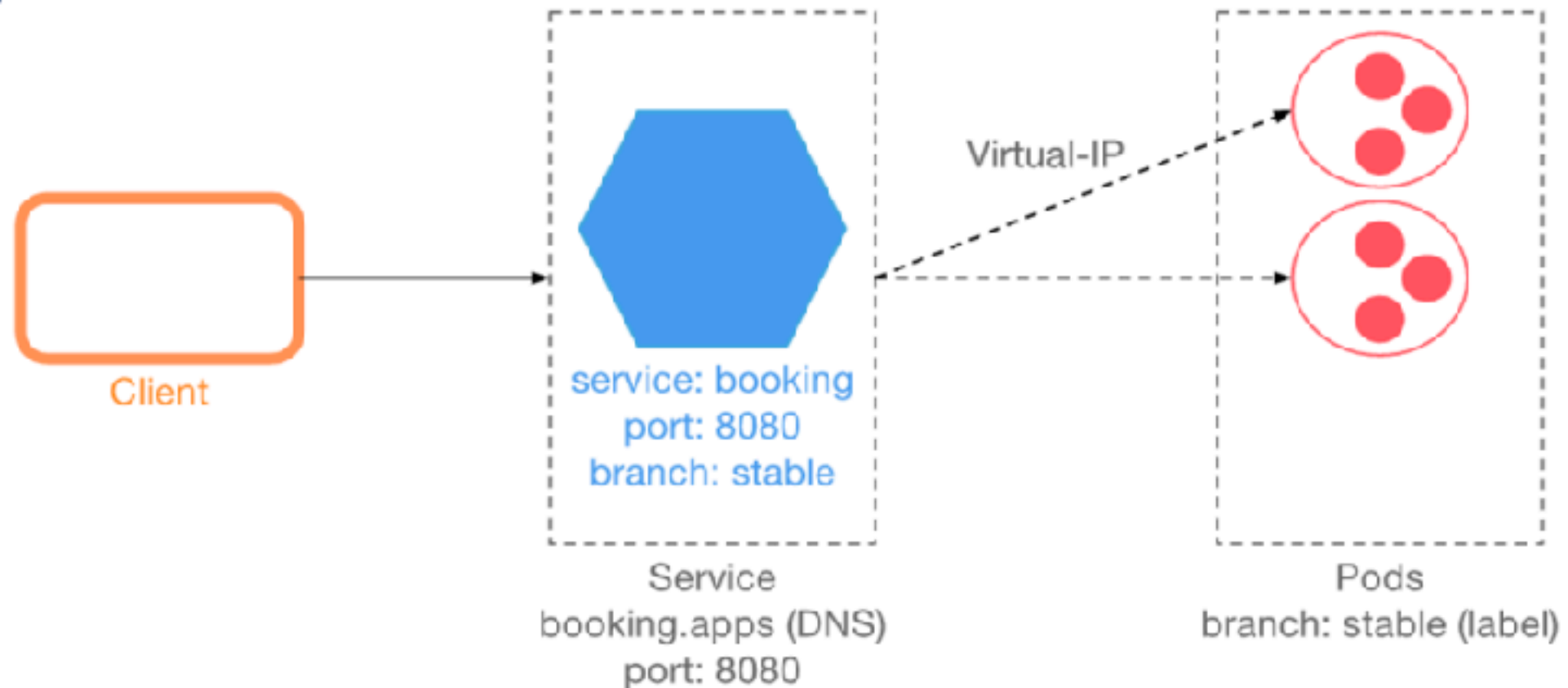
Container/Image boundary similar to class
How to use/manage a collection of container ?



Fault-tolerant by design

Design for failure

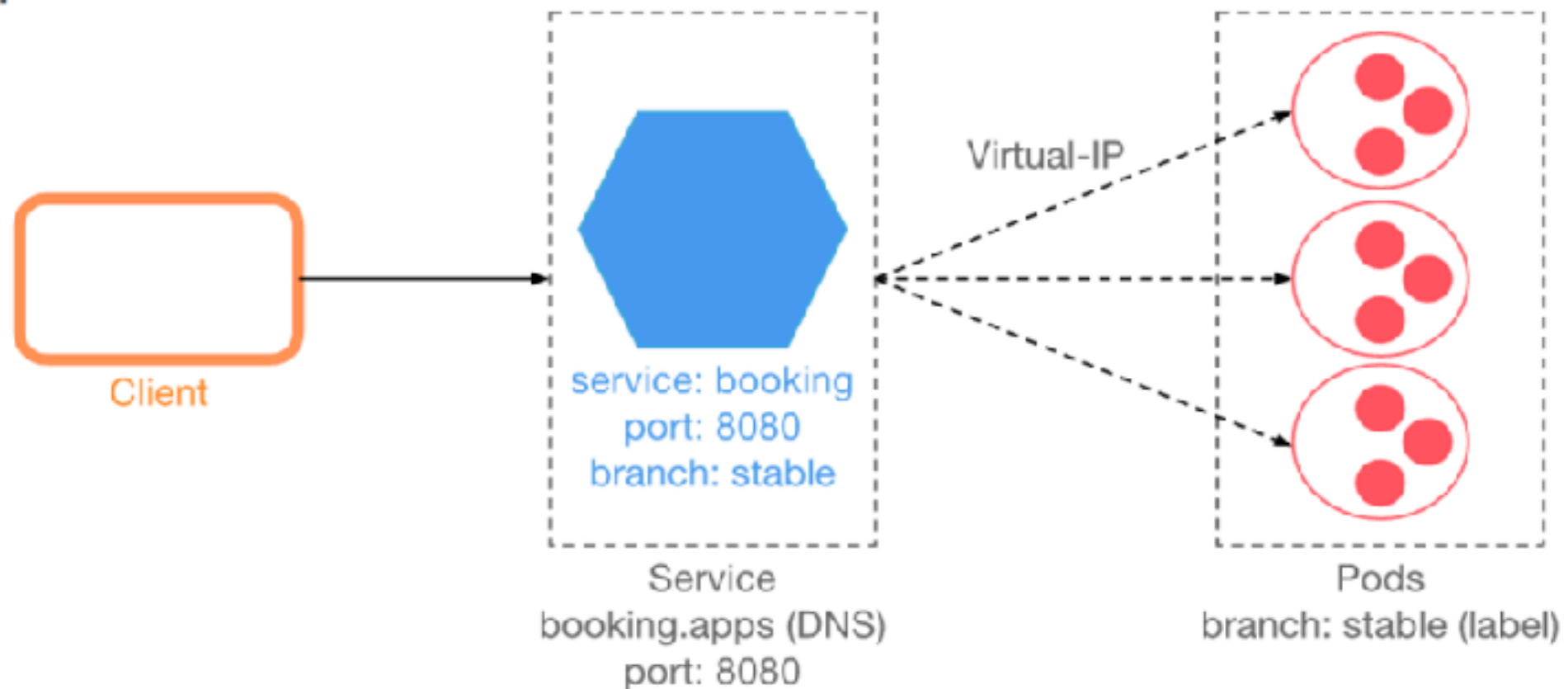
replicas = 2



Fault-tolerant by design

Design for failure

replicas = 3



Deployment, not infrastructure

Software deployment is hard

Infrastructure provisioning/re-provisioning

Configuration networking and load balance

Redundancy (scale-out)

Lifecycle management (Software update)

K8s support for deployment

Scale-out service

Rolling update for new version

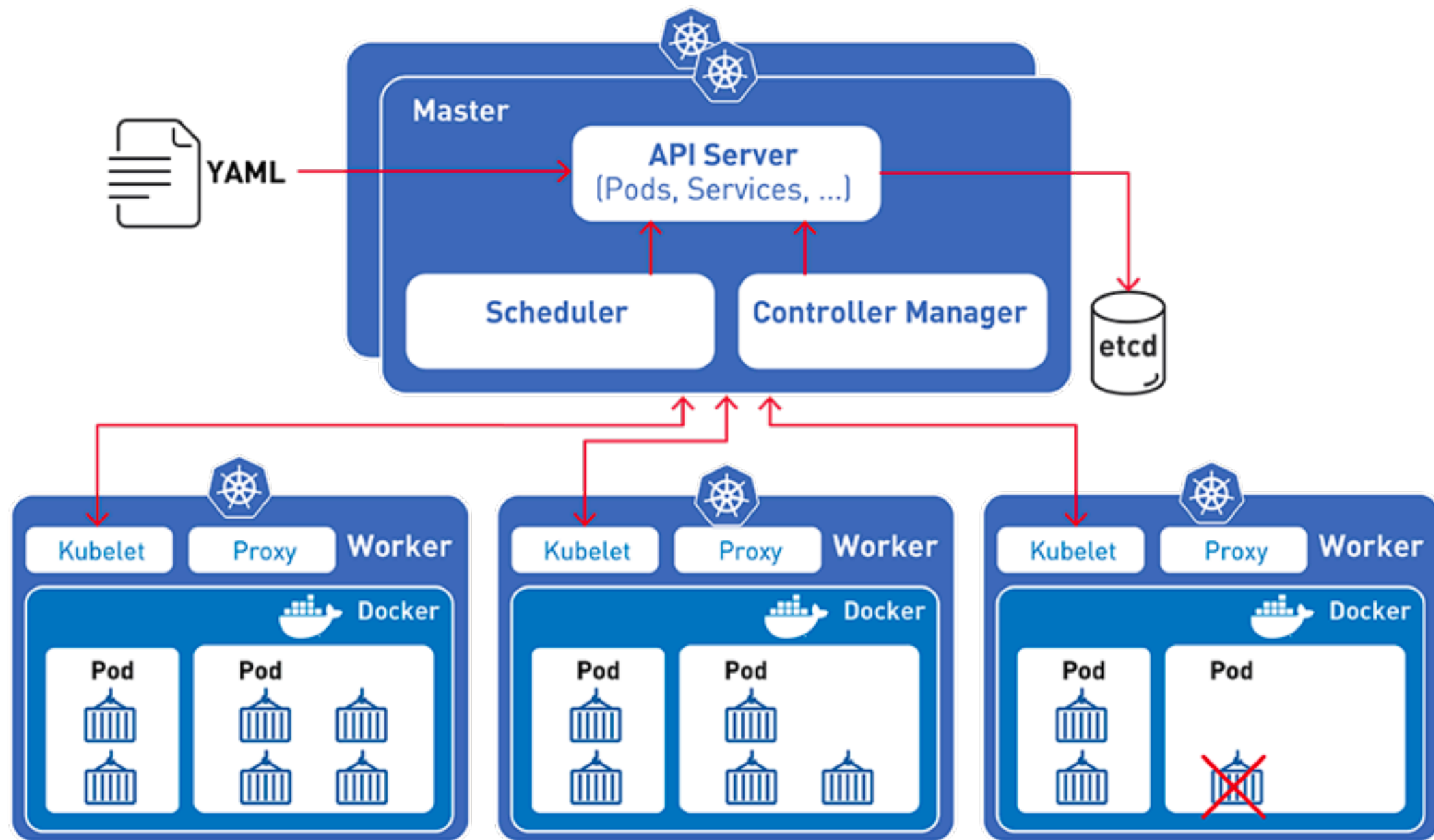
Rollback to a previous version

Pause and resume a deployment

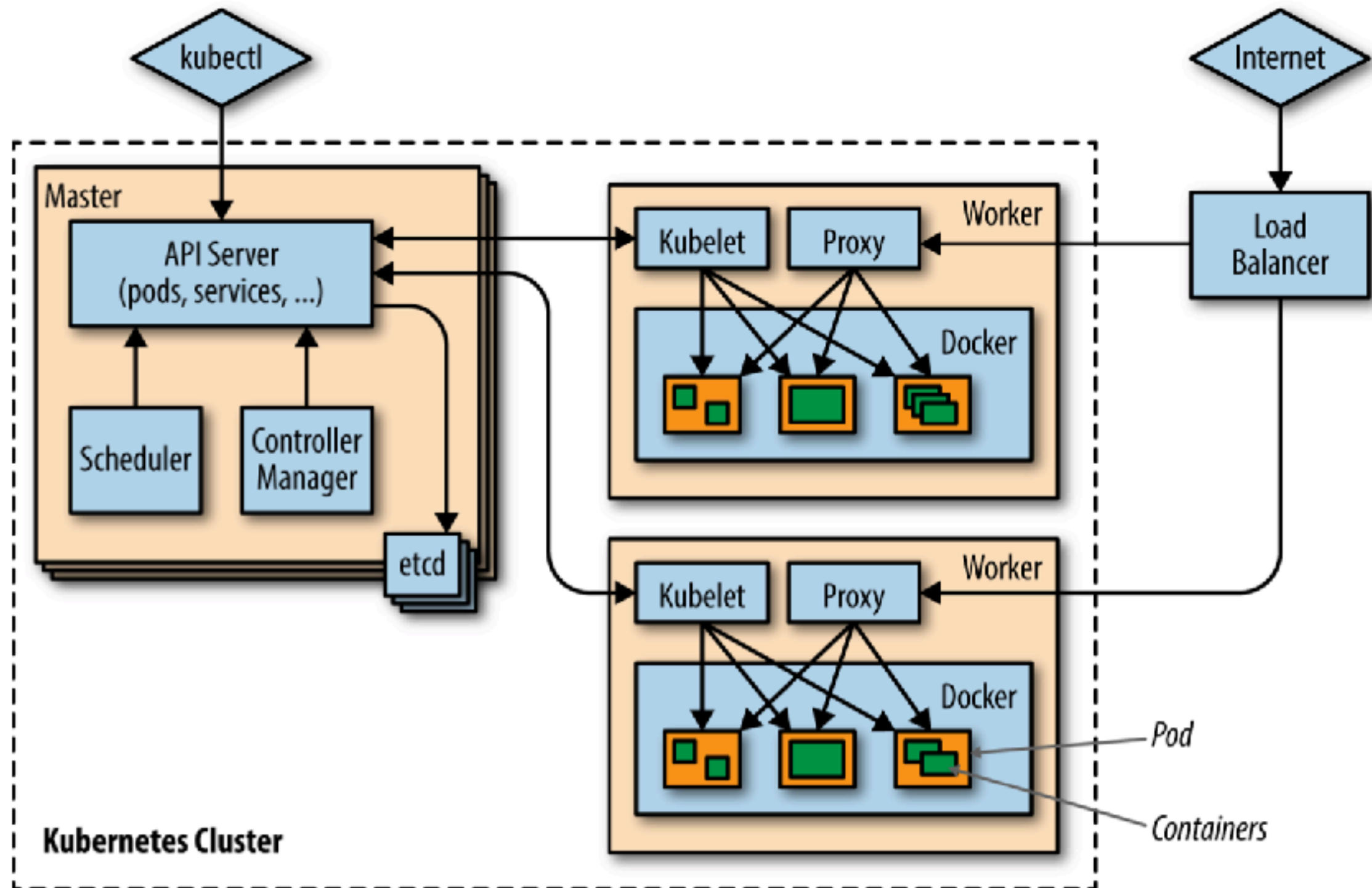
Horizontal auto-scaling

Canary deployment

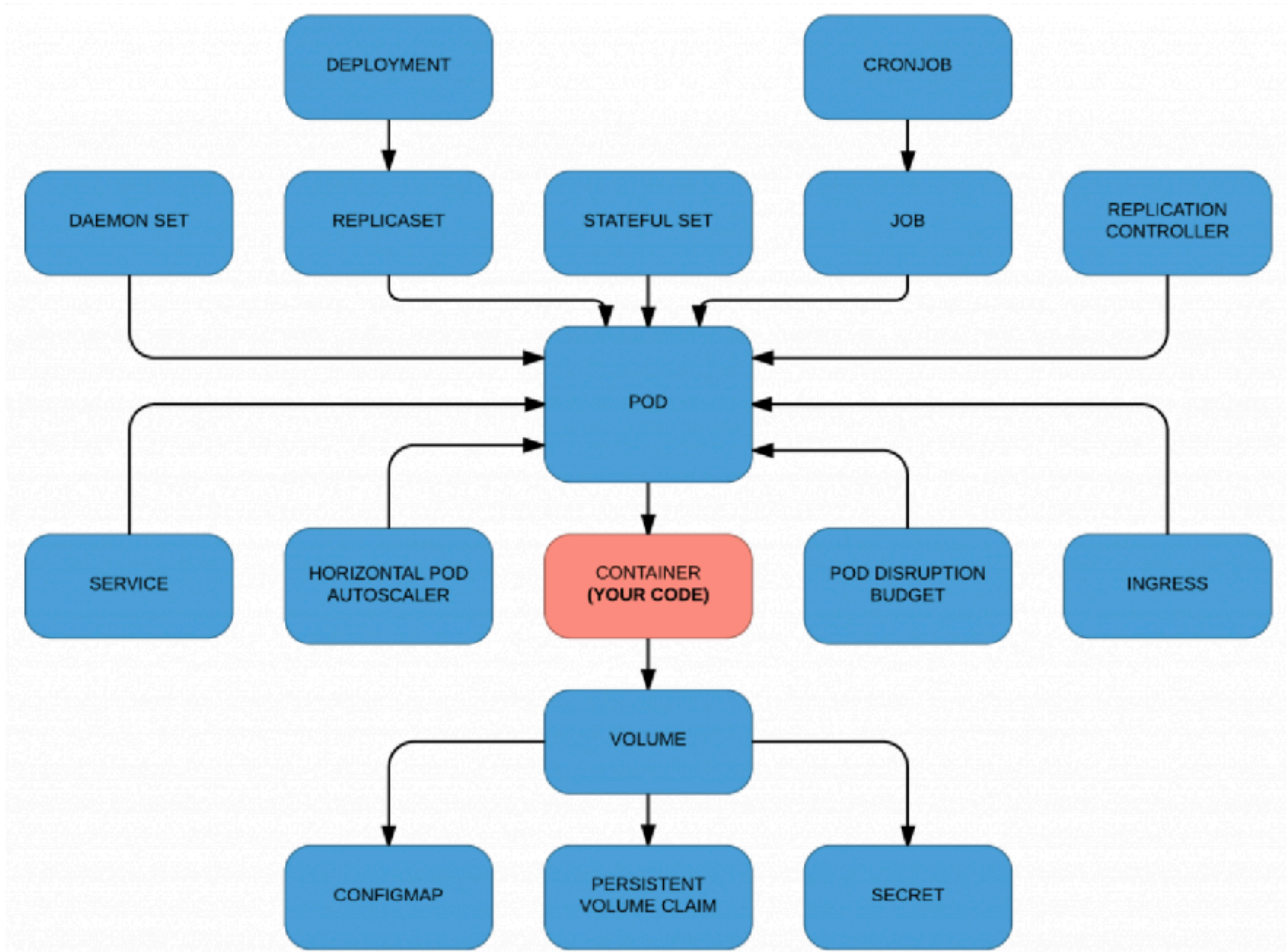
Kubernetes

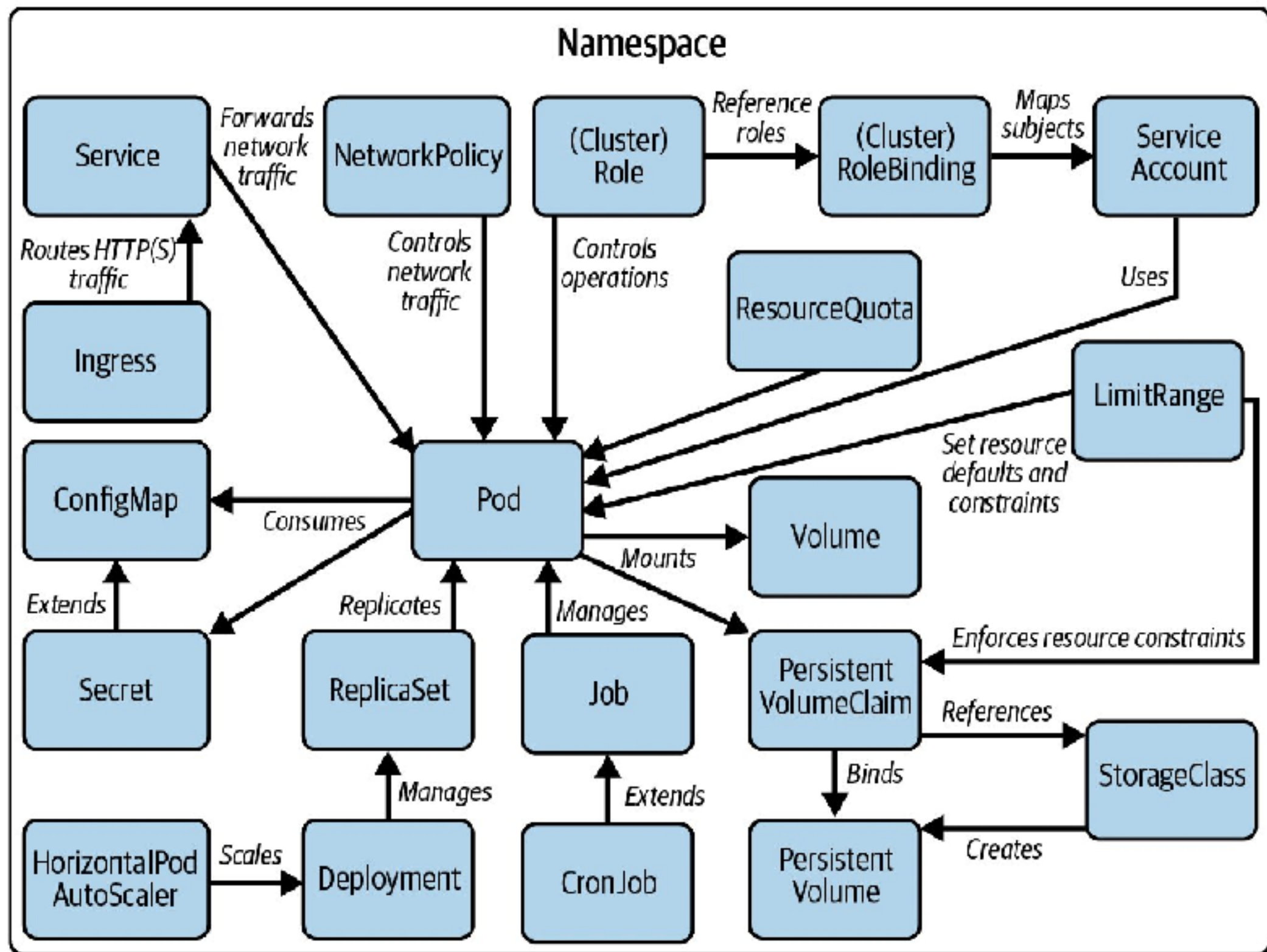


Kubernetes Architecture



K8s components





K8s components

Pods

Services

Deployments

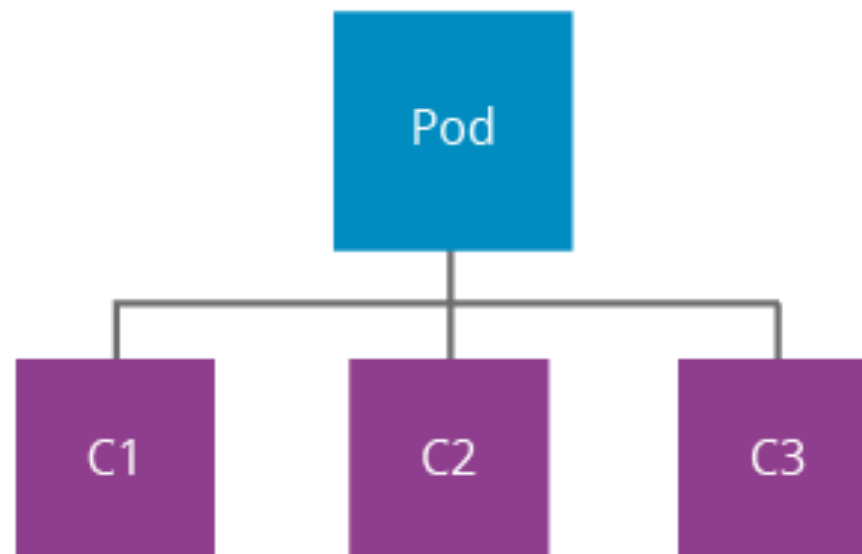
ConfigMap and Secret

Volumes

StatefulSets

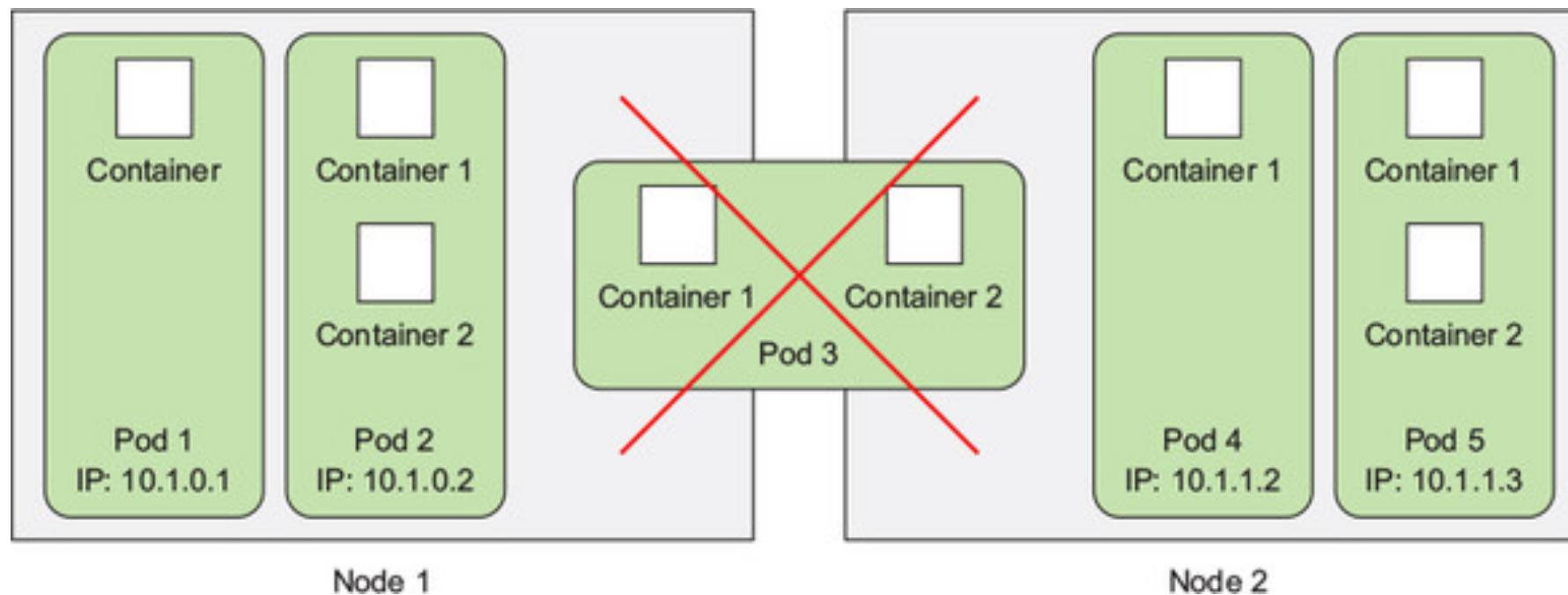
Pods

Small group of co-located containers
Optionally shared volume between containers
Basic deployment unit in Kubernetes



Pods

1 pods = 1 container
1 pods = N containers



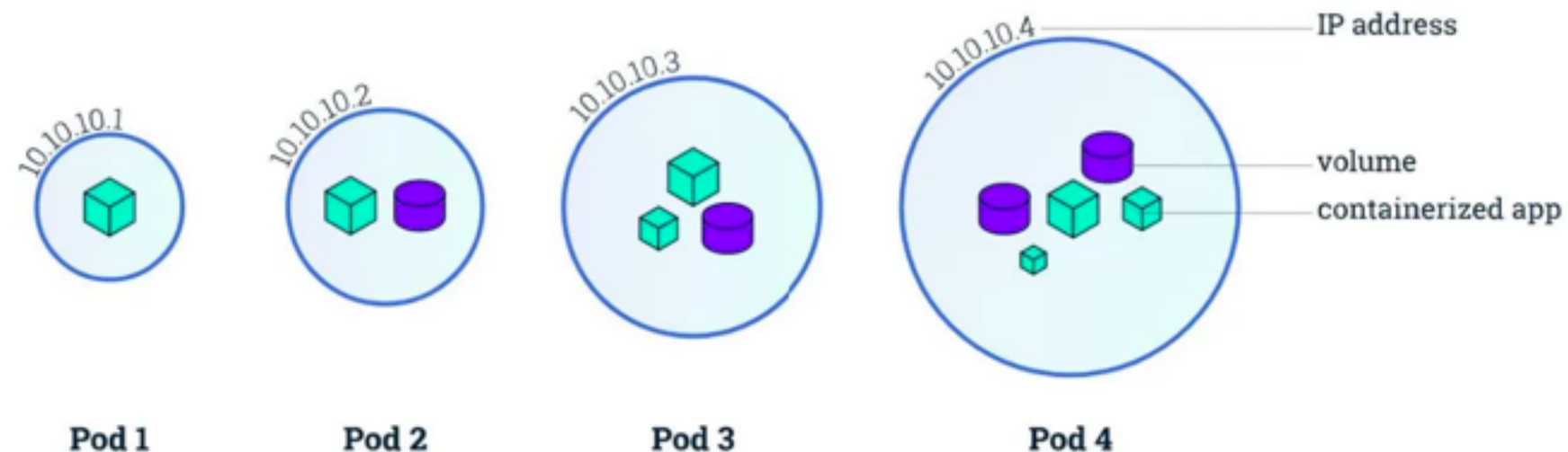
All containers in same Pods

Share process ID

Share network interface

Share Hostname/IP/Port

Share Unix Time Sharing (UTS)

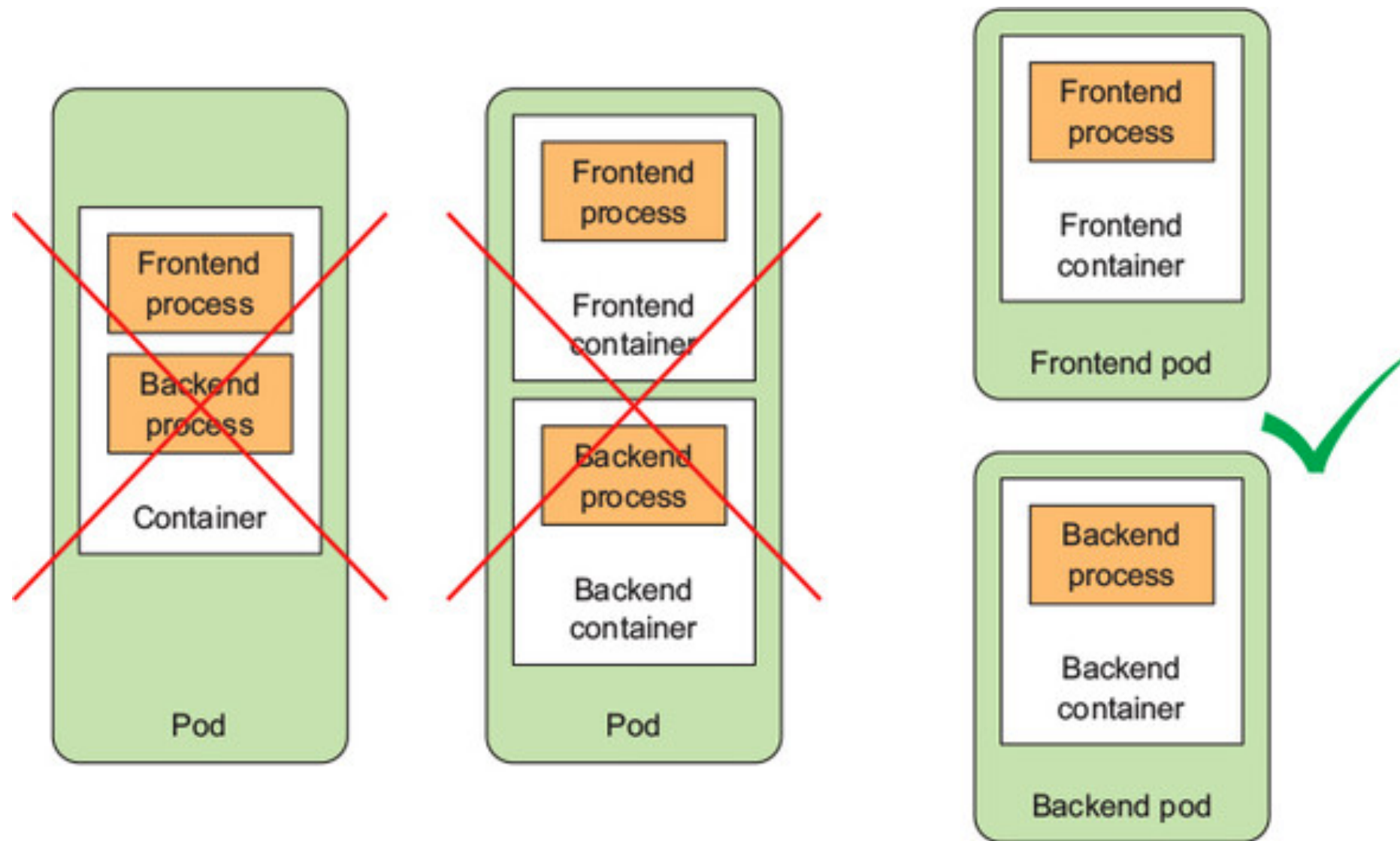


Pods Life Cycle

Phase	Description
Pending	Accepted by kubernetes but container not created yet
Running	Pods bound to the node, all containers created and at least one container is running/starting/restarting
Successed	Containers exited with status 0
Failed	All containers exit and at least one exited with non-zero status
Unknow	State of Pods can't be determined due to communication issues with its node

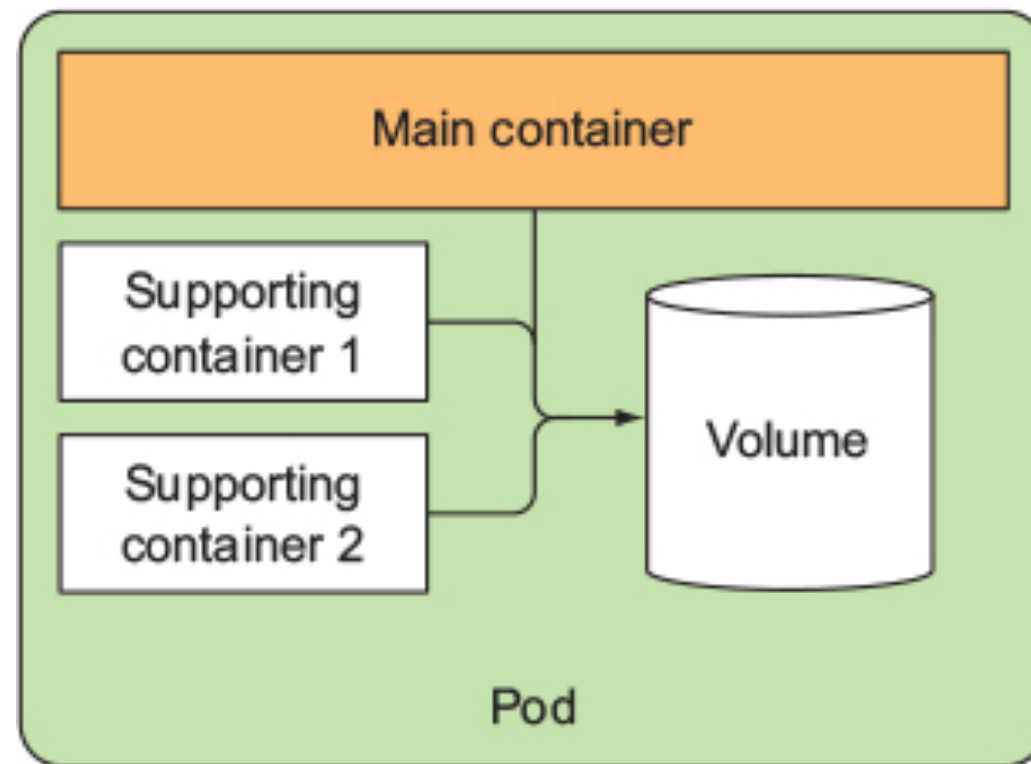
Organize container across Pods

Split multi-tiers app into multiple pods

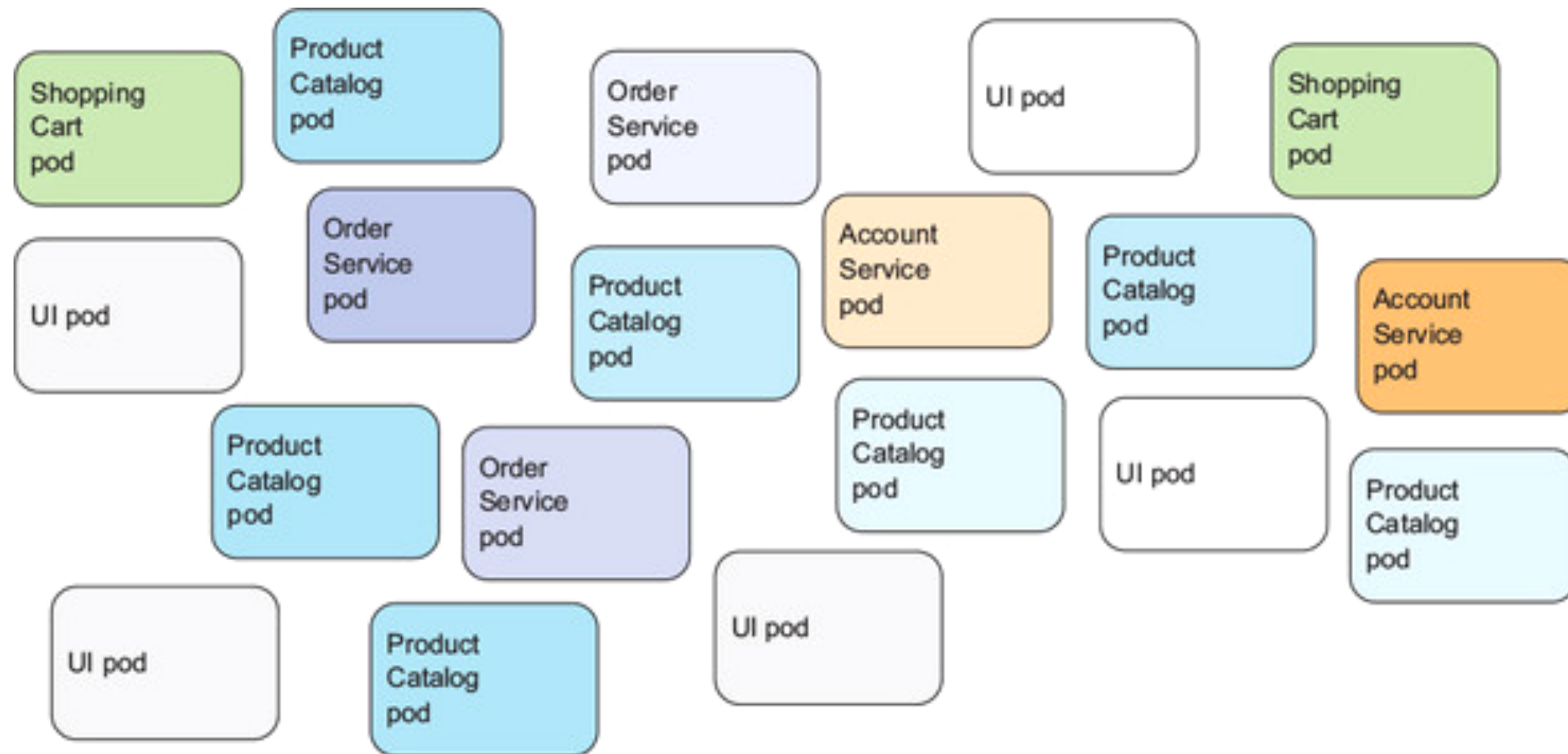


Organize container across Pods

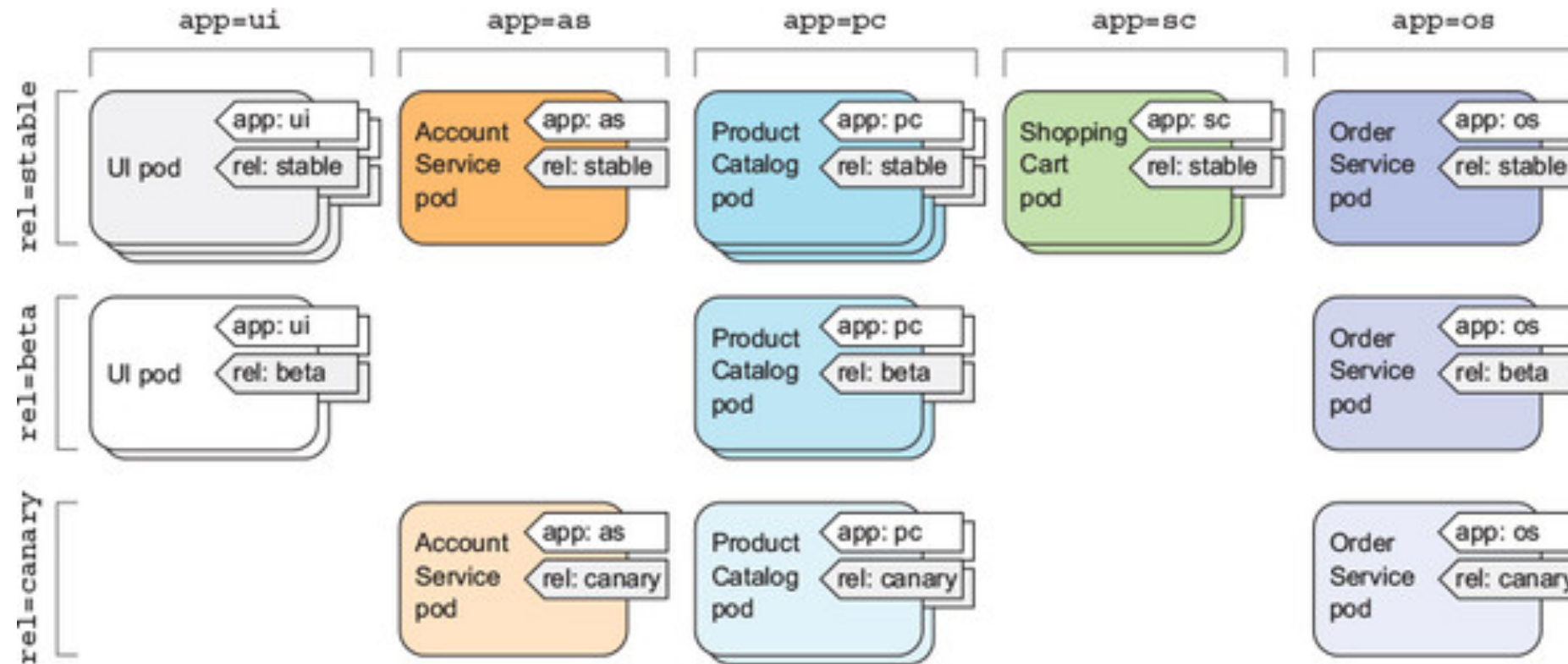
When to use multiple containers in a pods



Organize pods with Labels

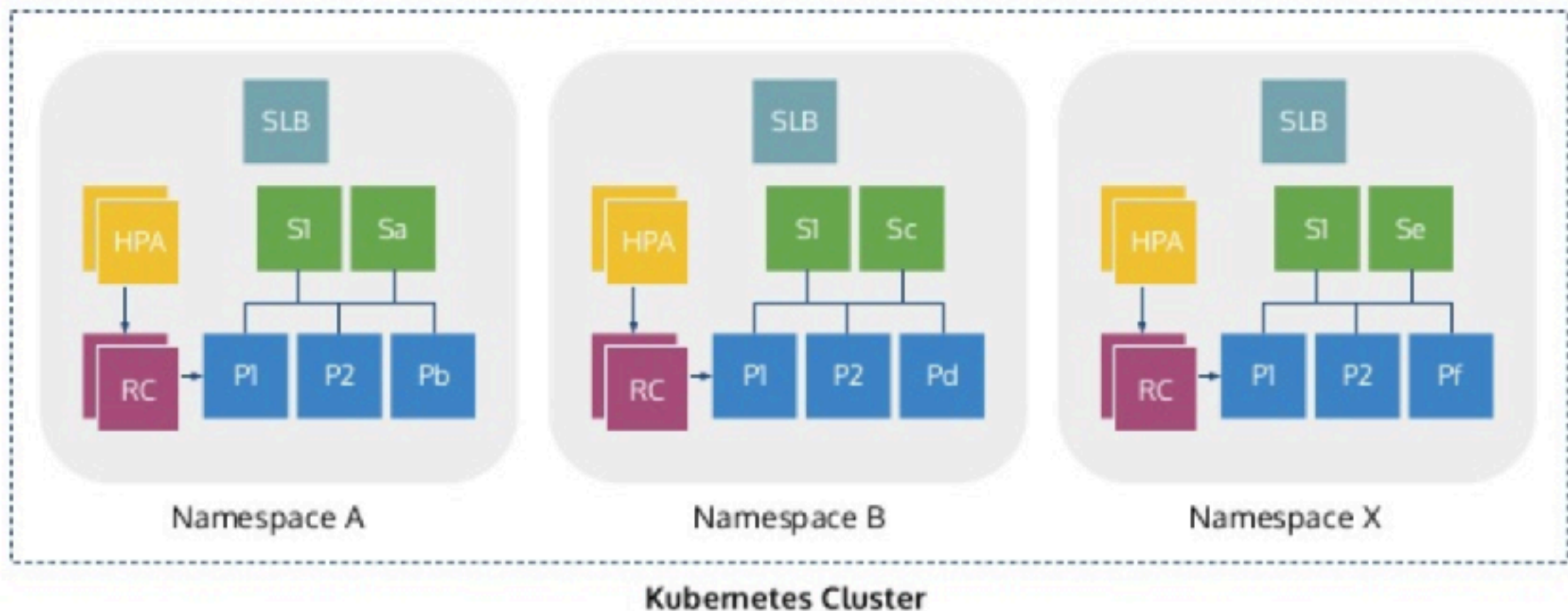


Organize pods with Labels



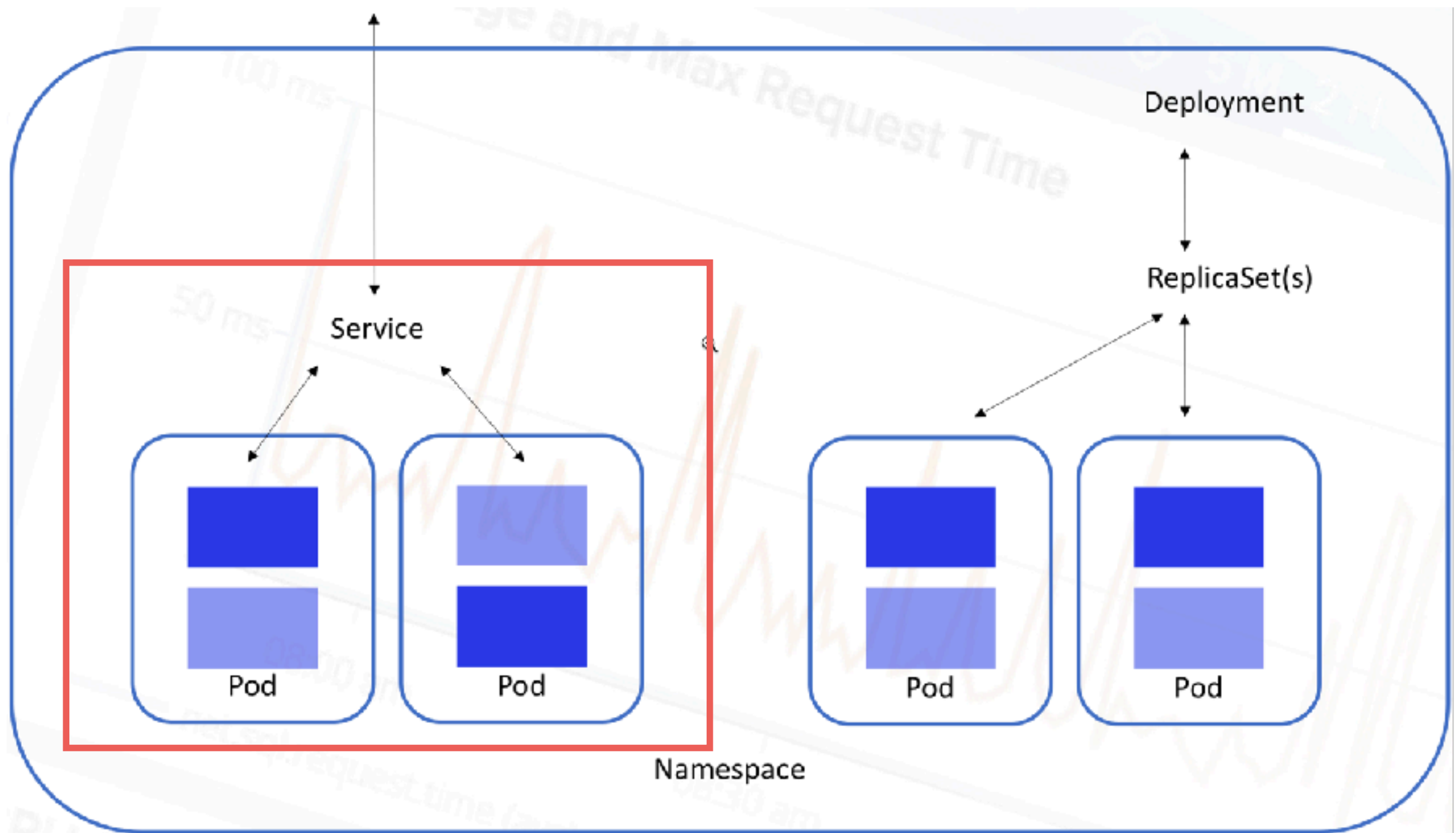
Pods Namespaces

Allow different teams to use the same cluster

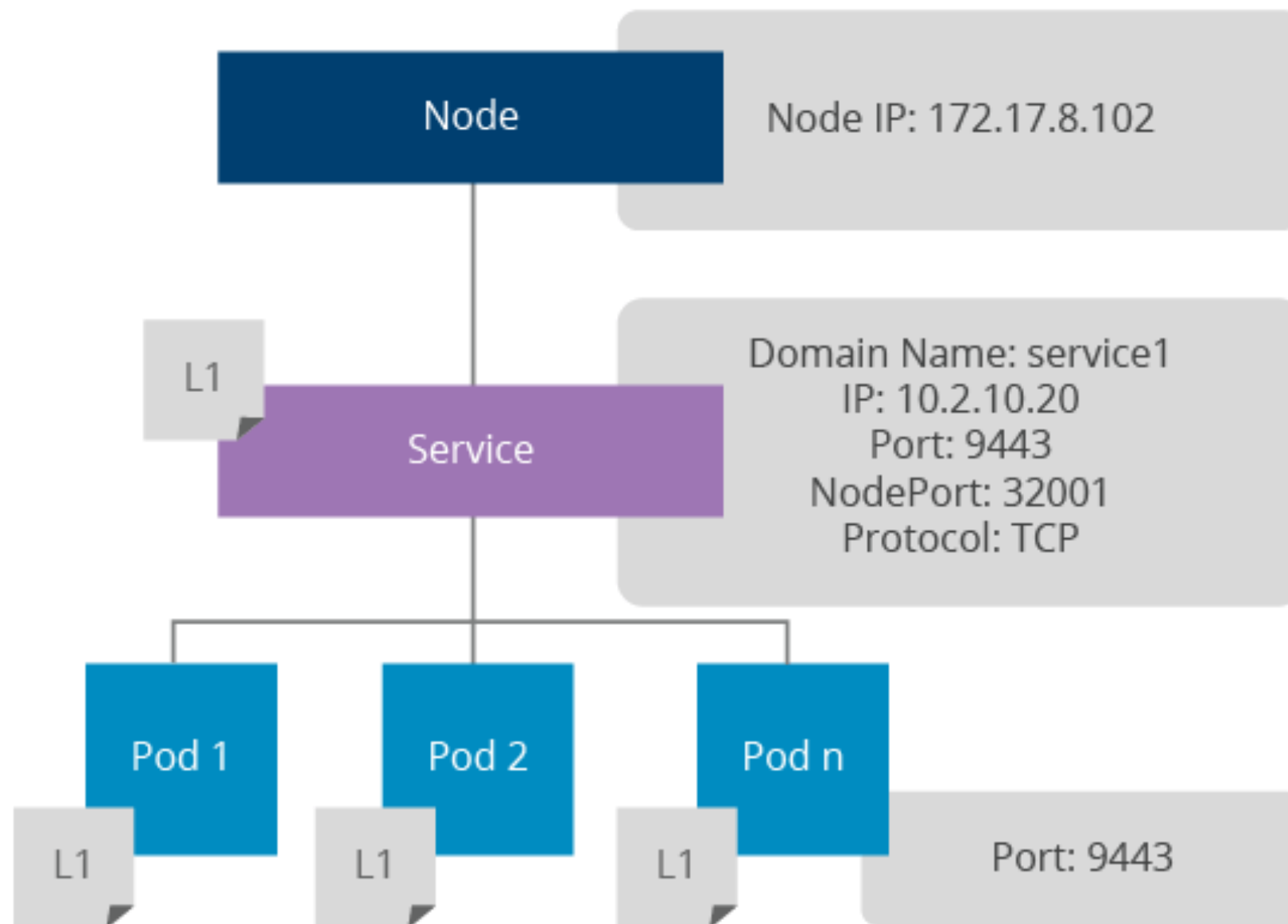


Services discovery and Load balancing

Services



Services



Services

Independent from Pods

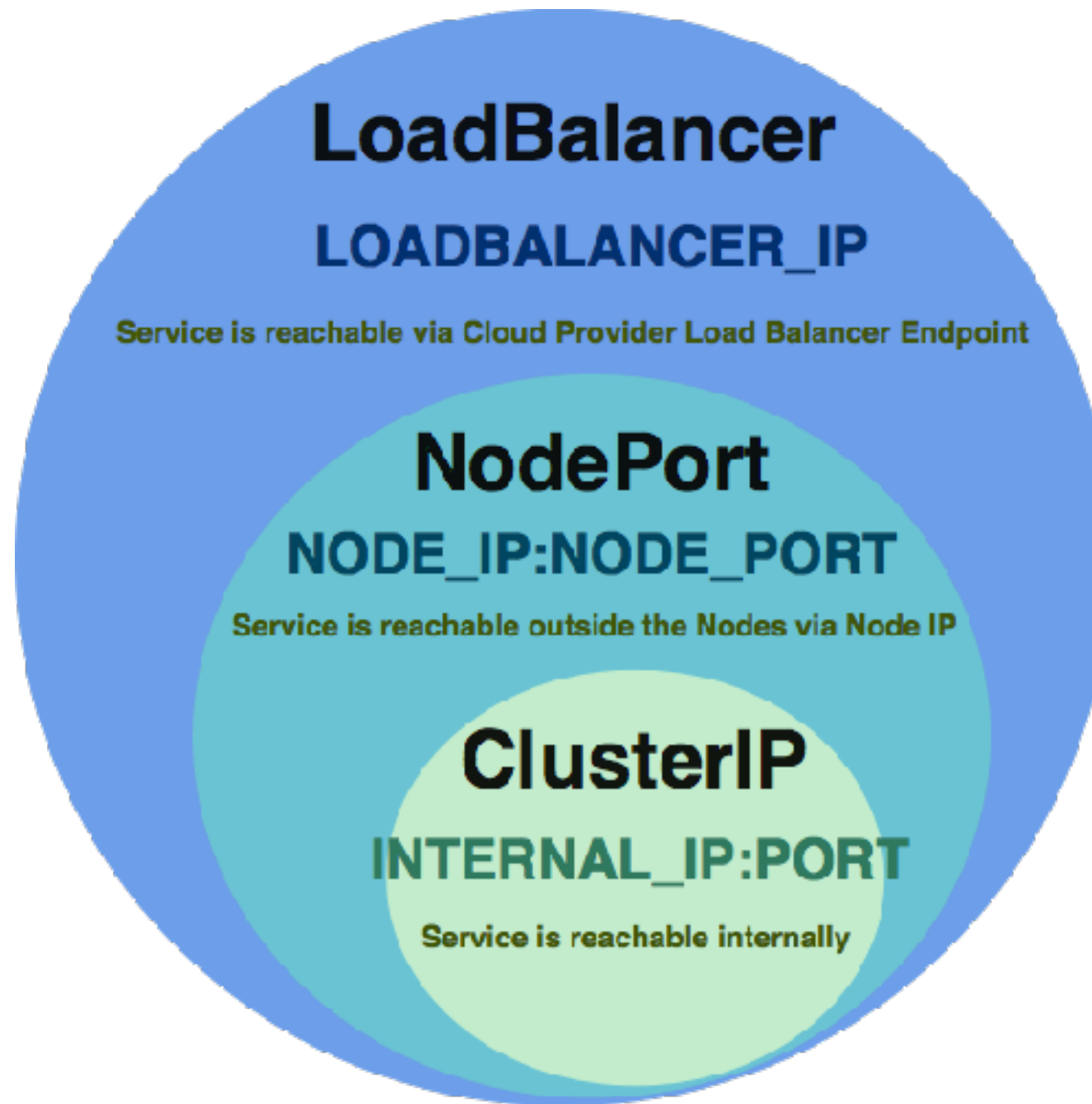
Abstraction layer of Pods

Provide load balance

Expose access Pods/Load balance

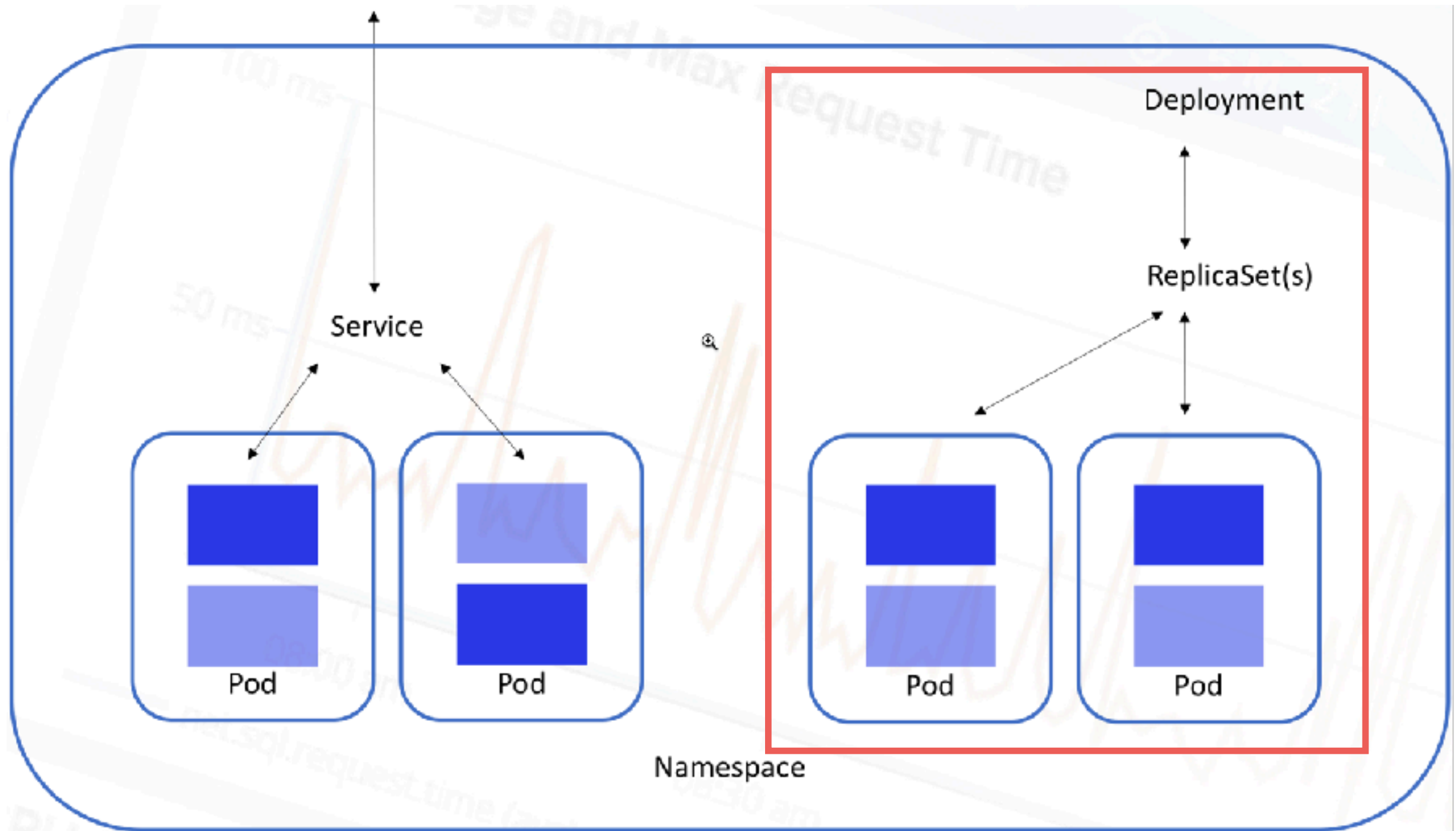
Find Pods by label selector

3 types of services

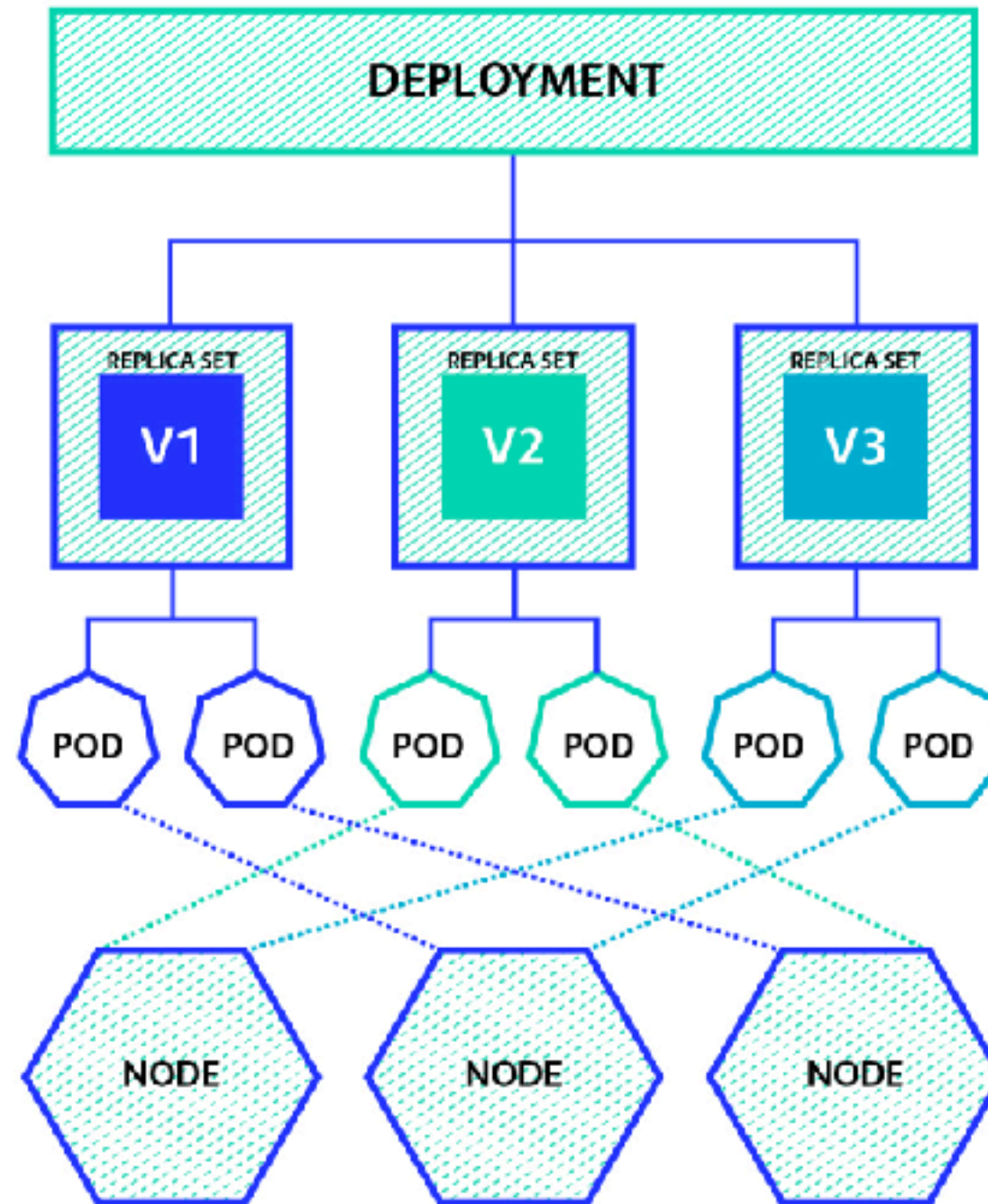


Deployment and ReplicaSet (RS)

Deployment and ReplicaSet



Deployment and ReplicaSet



Deployment and ReplicaSet

Next-generation of Replication Controller

Provide function to maintain versioning of Pods

Update new version (Rollout)

Revert to old version (Rollback)

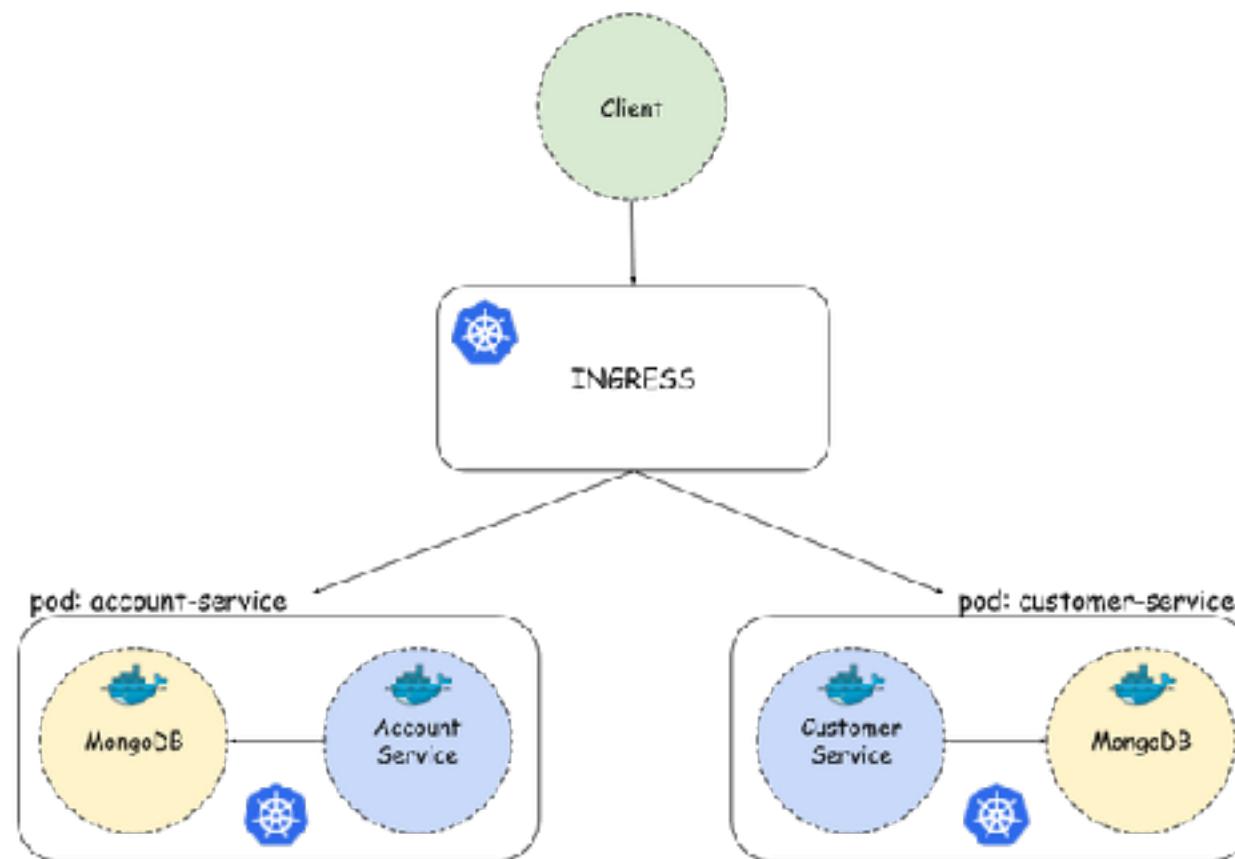
Scale a deployment

Pause/Resume process

Ingress network

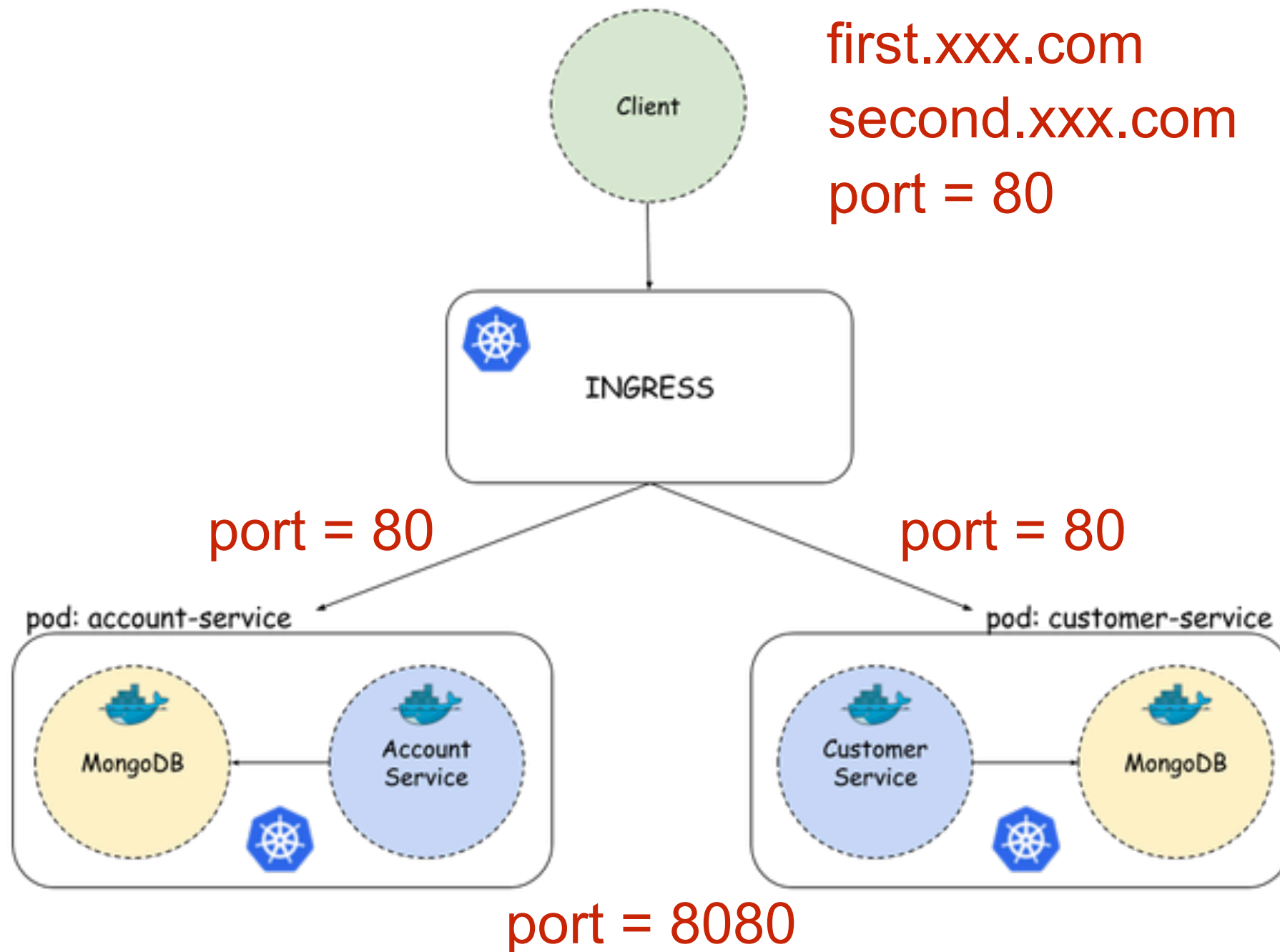
Ingress Network

How to handle multiple services in same port ?
How to limit protocol to access ?



<https://kubernetes.io/docs/concepts/services-networking/ingress/>

Ingress Network

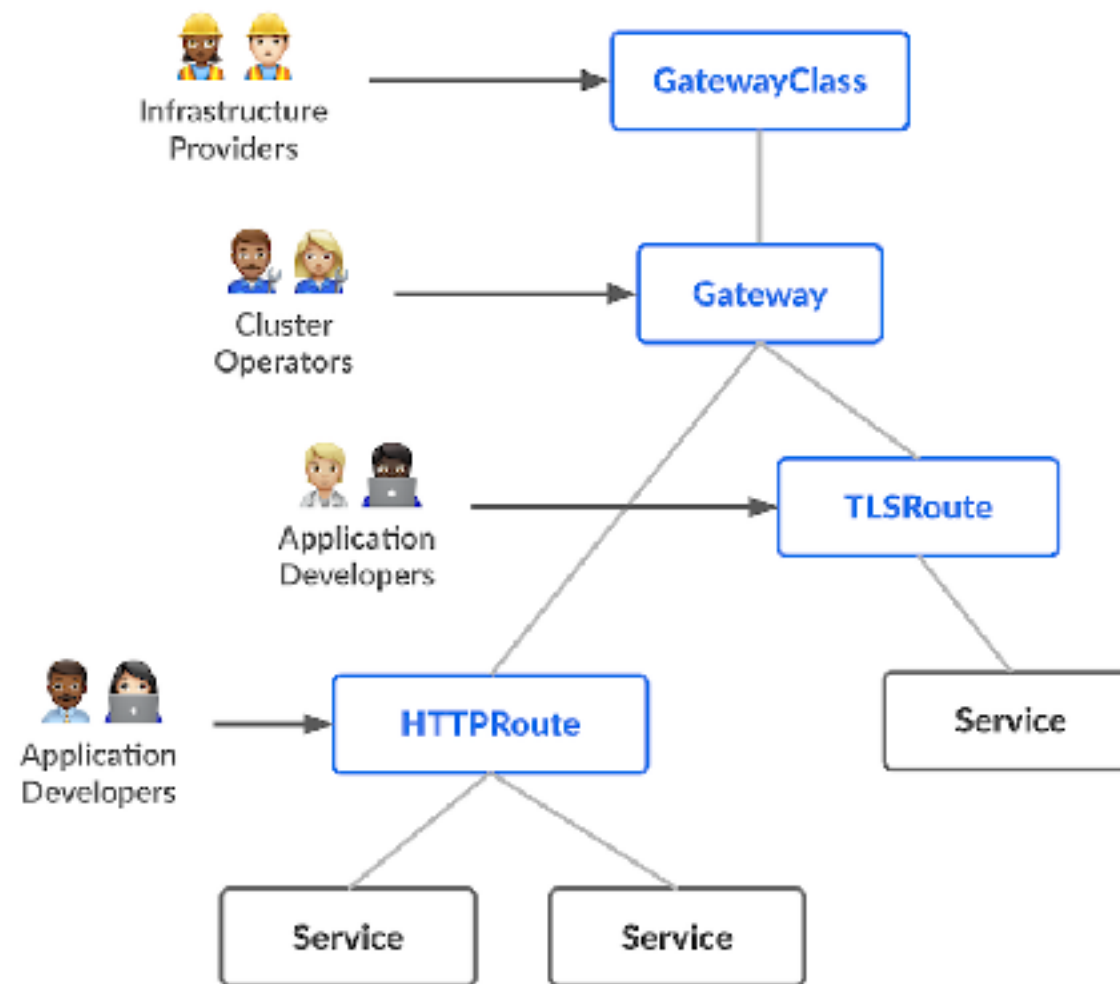


Gateway API

<https://kubernetes.io/docs/concepts/services-networking/gateway/>

Gateway API

Dynamic infrastructure provisioning
Advance traffic routing



<https://gateway-api.sigs.k8s.io/>

Gateway API

Request flow of HTTP traffic



<https://gateway-api.sigs.k8s.io/>

StatefulSet

StatefulSet

Bringing the concept of ReplicaSets to **stateful** Pods

Enable running Pods in **cluster** mode

Ideal for deploy **highly** available database **workload**

StatefulSet

Stable, unique network identifiers

Stable, persistent storage

Ordered, graceful deployment and scaling

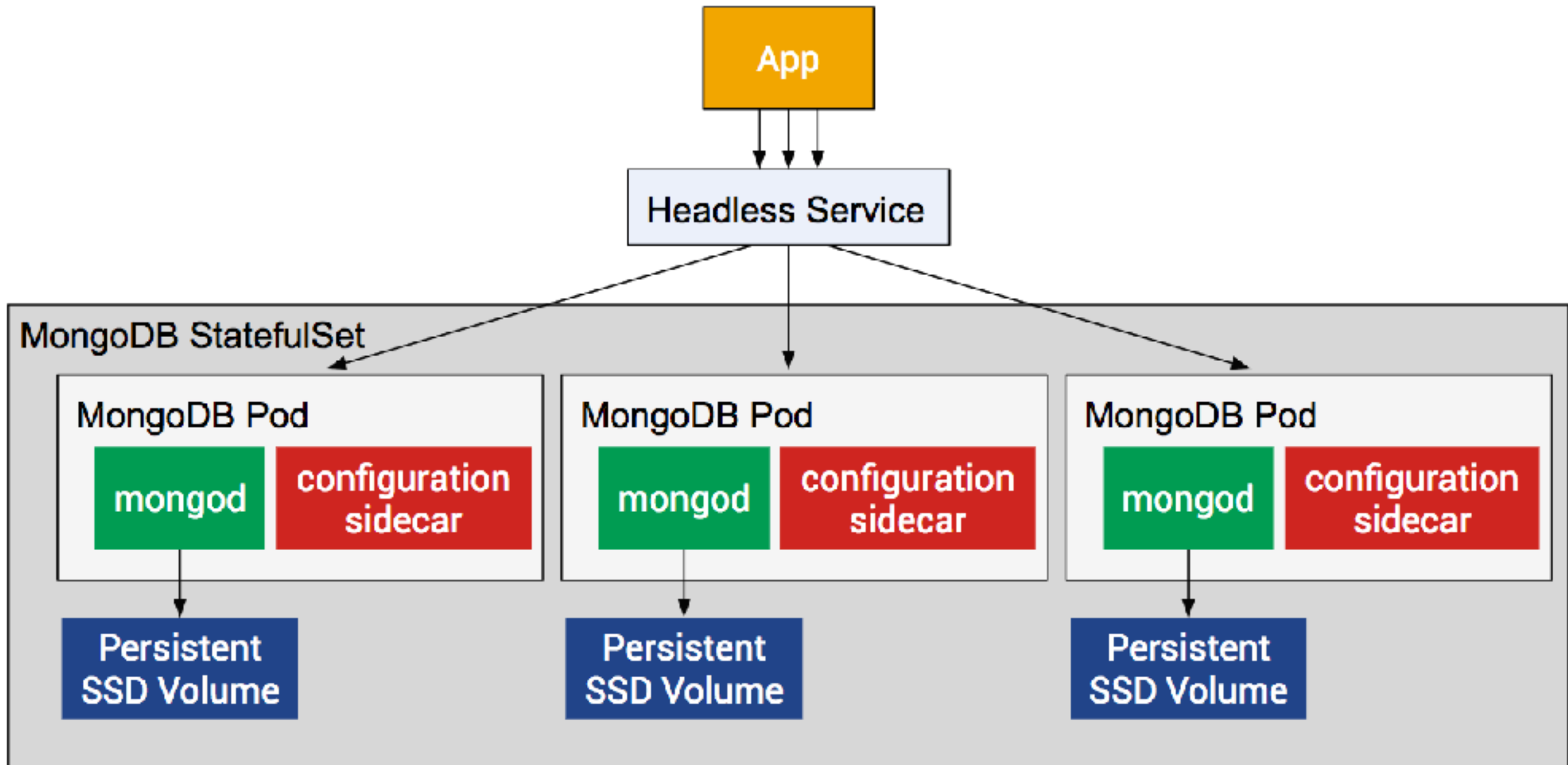
Ordered, graceful deletion and termination

Key concepts

Pods are created sequentially

Pods are terminated in LiFo (Last in, First out)

StatefulSet

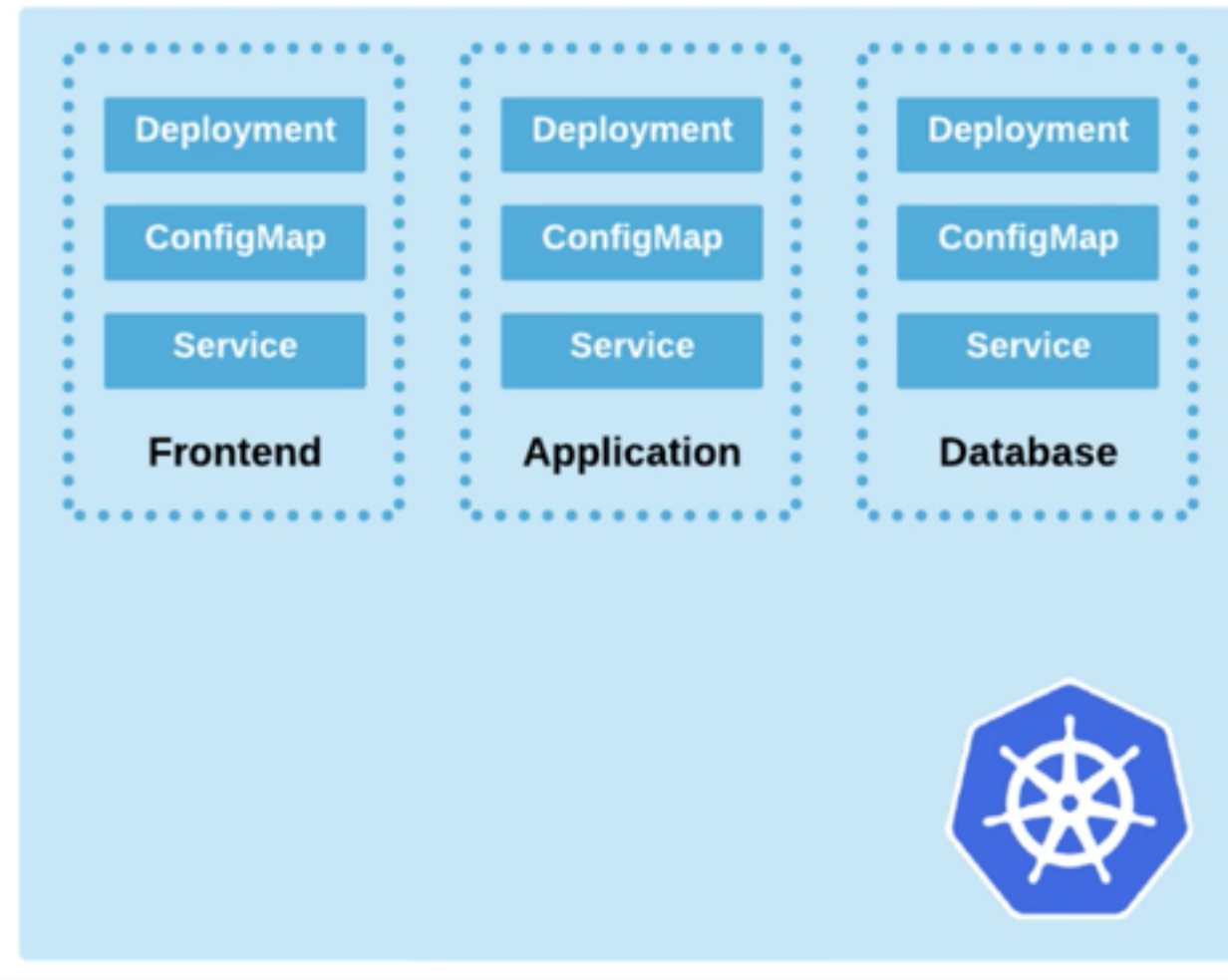


Workshop

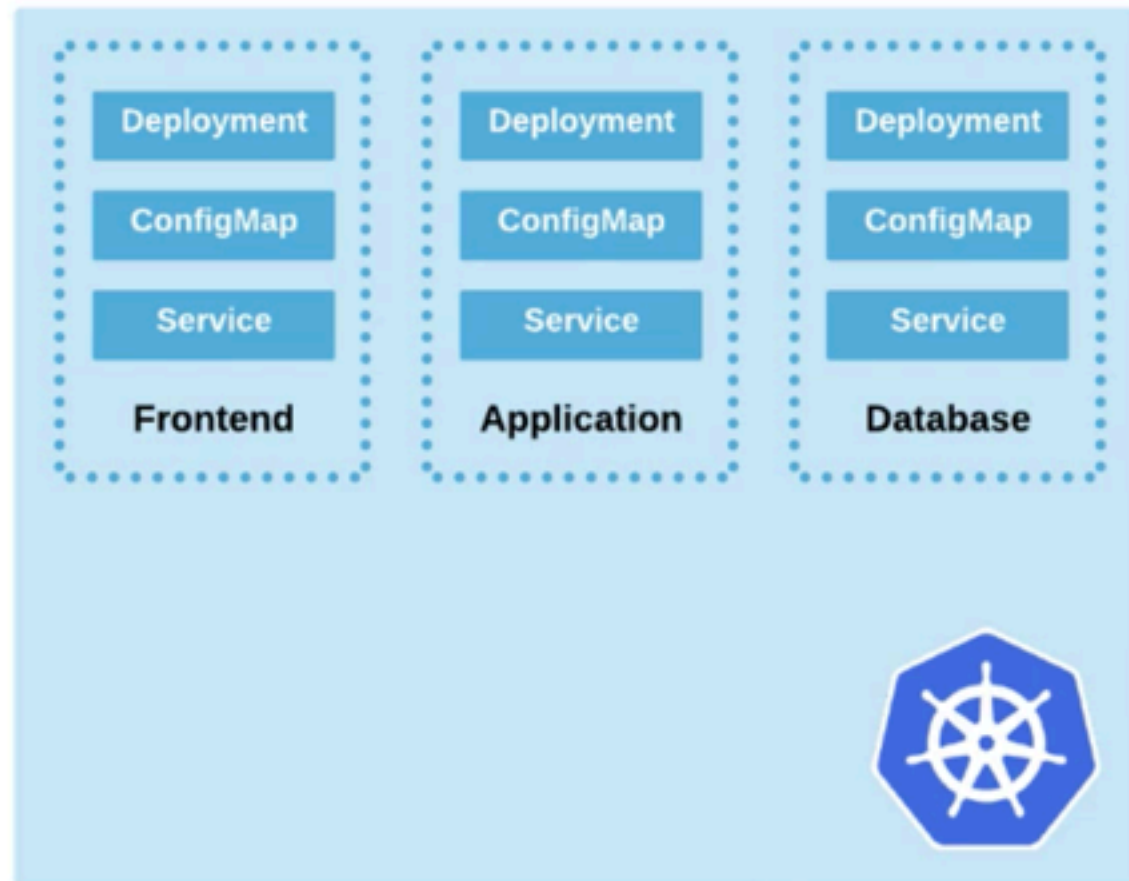


kubernetes

K8s resources



K8s resources



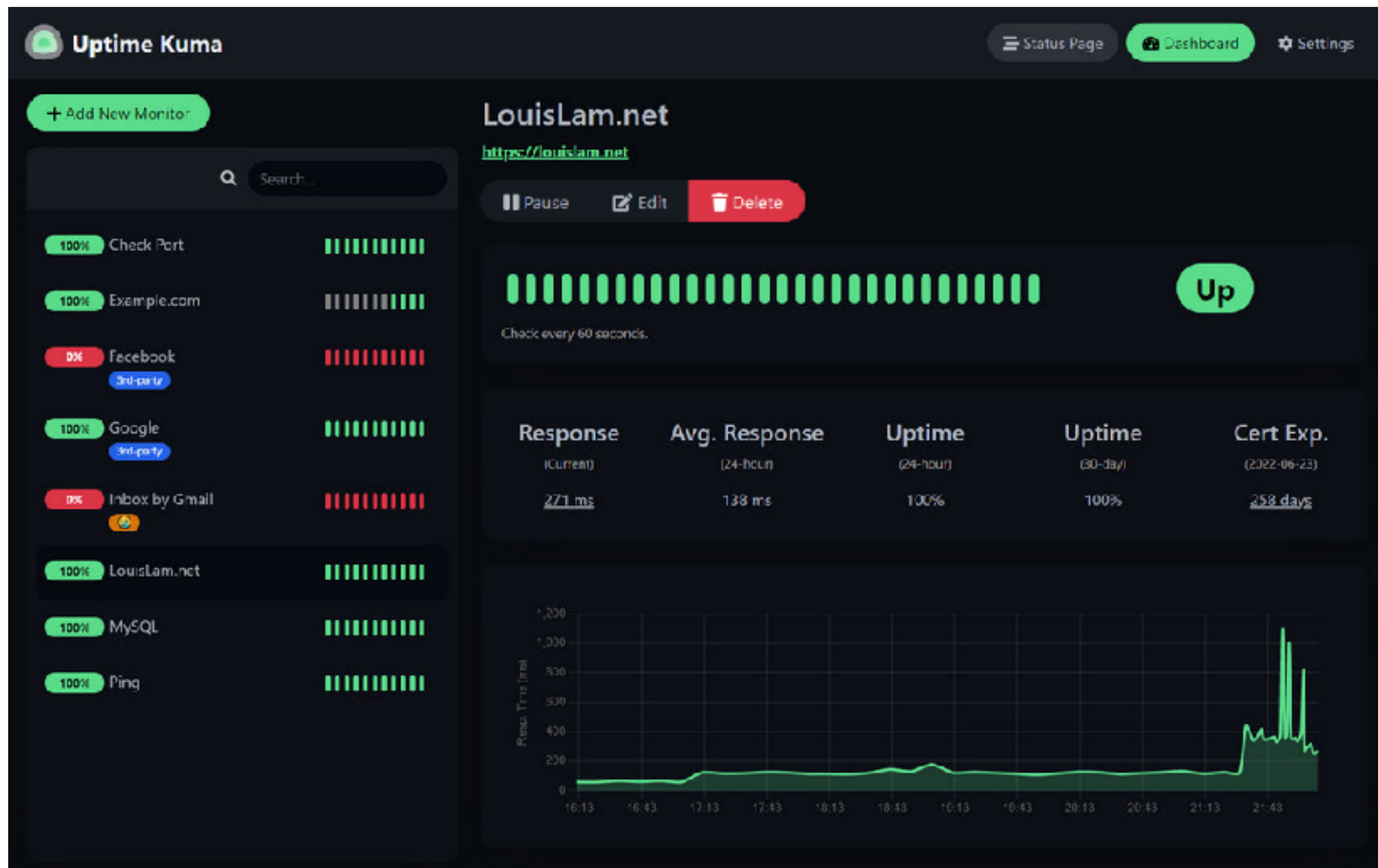
```
~/demo: kubectl apply -f  
~/demo: kubectl apply -f  
~/demo: kubectl apply -f
```

```
~/demo: kubectl apply -f  
~/demo: kubectl apply -f  
~/demo: kubectl apply -f
```

```
~/demo: kubectl apply -f  
~/demo: kubectl apply -f  
~/demo: kubectl apply -f
```

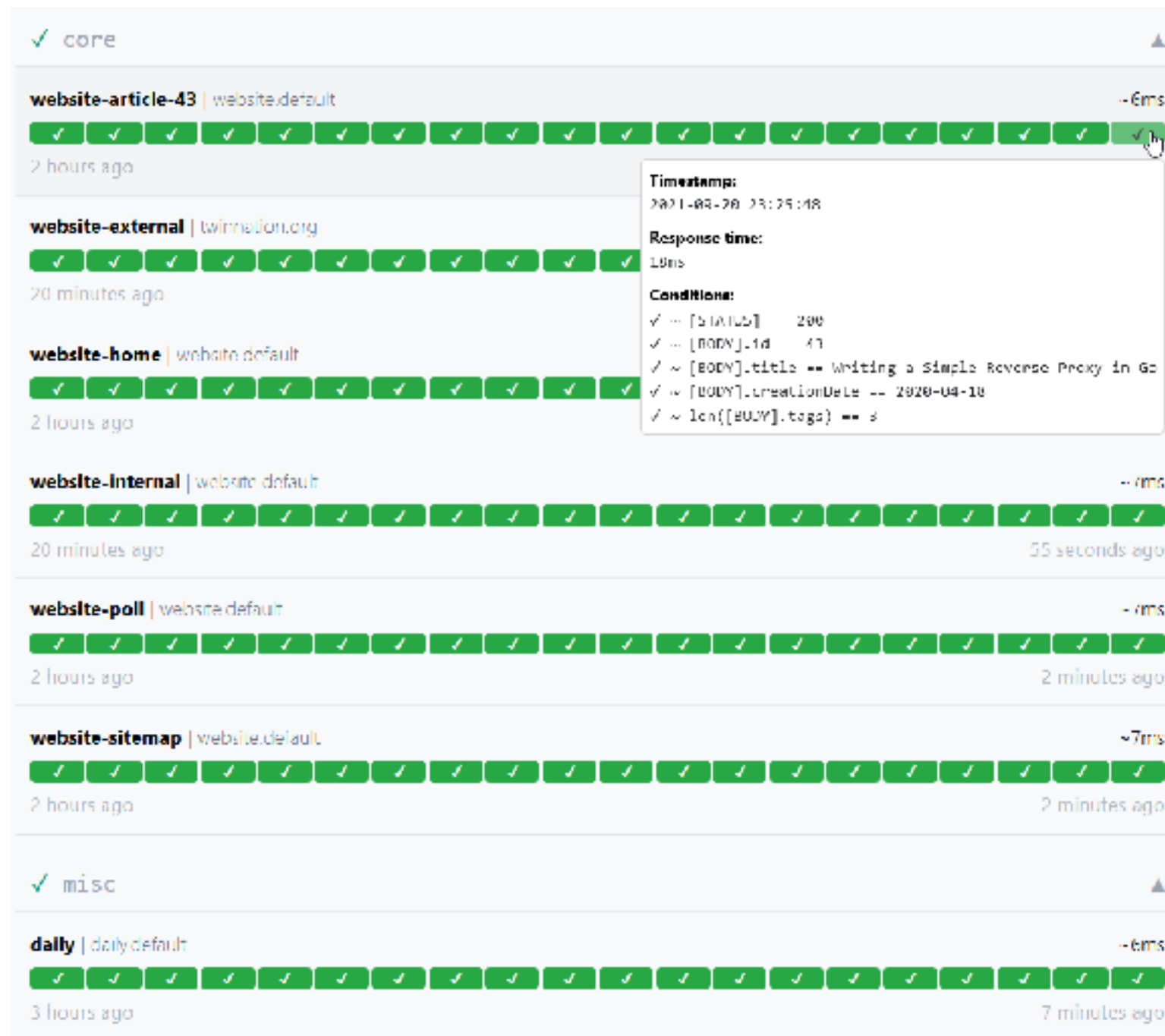
Observability Monitoring

Uptime Kuma



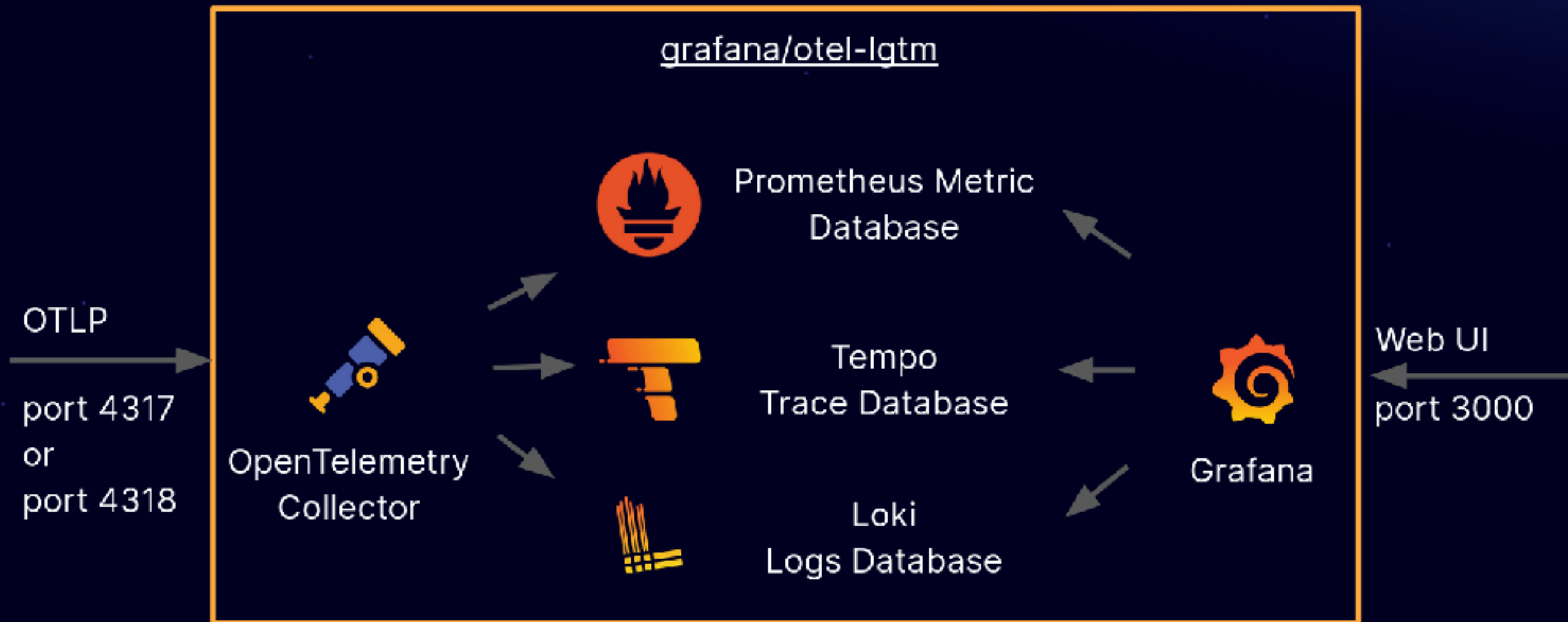
<https://github.com/louislam/uptime-kuma>

Gatus



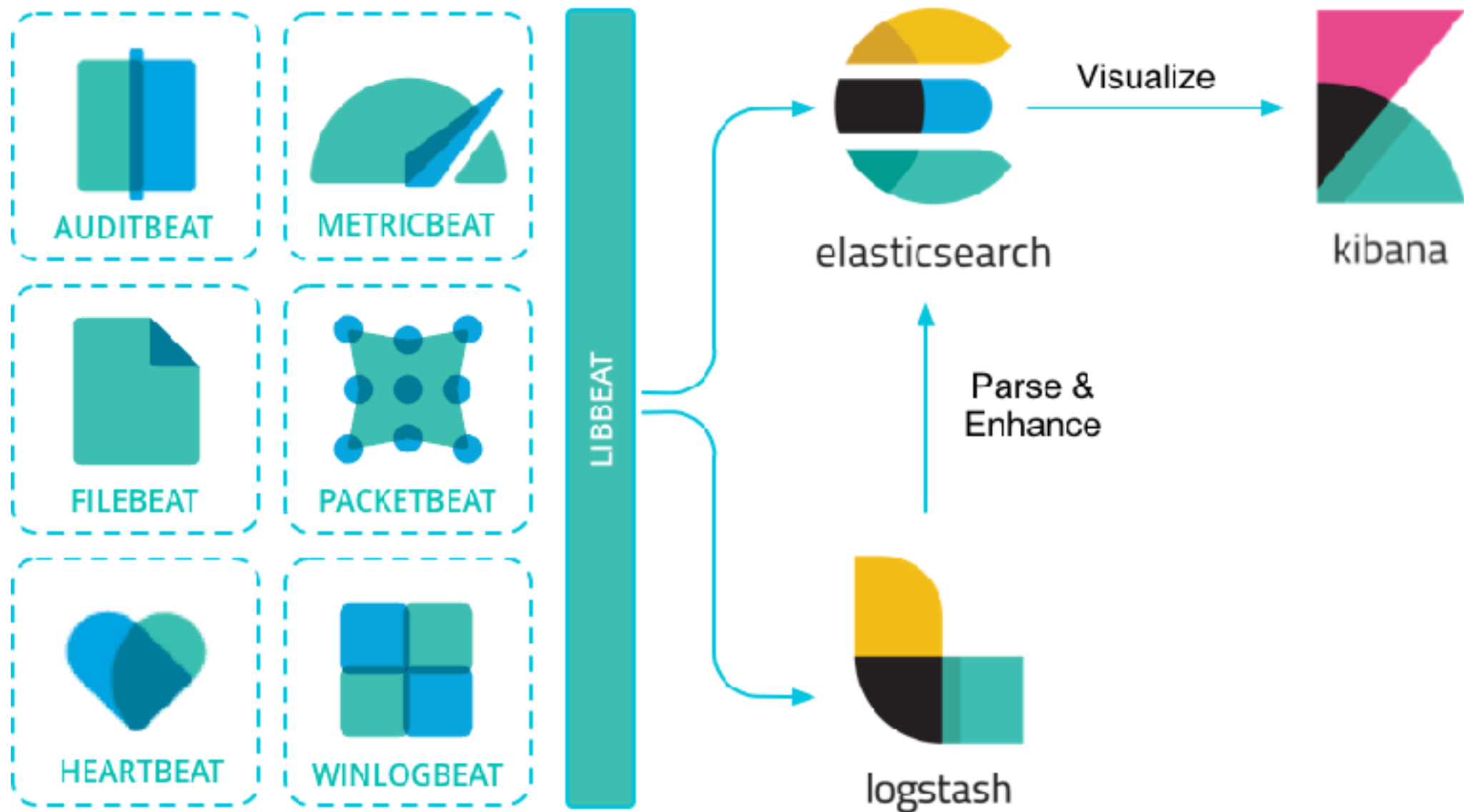
<https://gatus.io/>

LGTM Stack



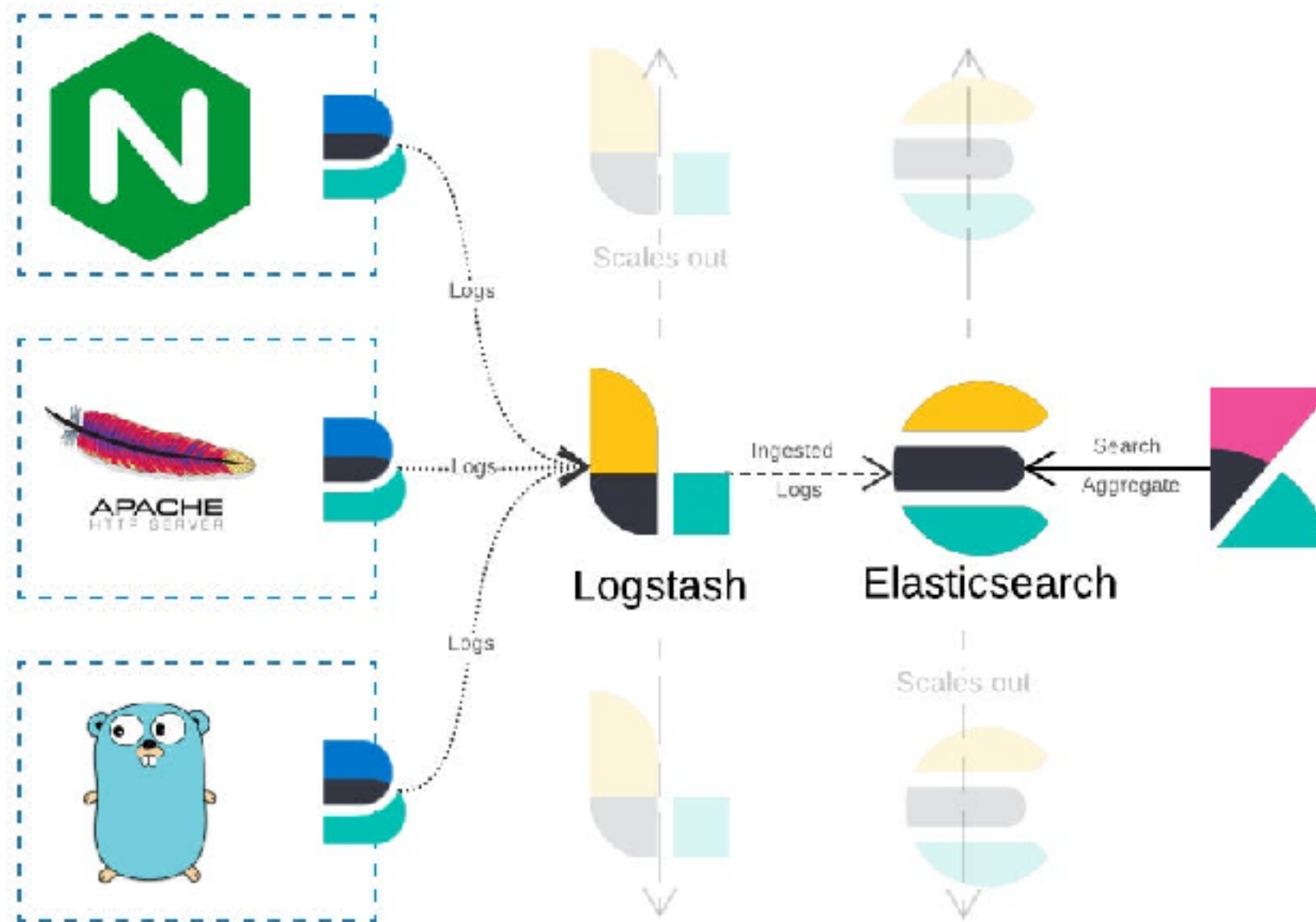
<https://github.com/grafana/docker-otel-lgtm/>

ELK Stack



<https://www.elastic.co/elastic-stack>

ELK Stack



<https://www.elastic.co/elastic-stack>